Bootstrapping over Free $\mathcal{R}$-Module

Ruida Wang[1,2], Jikang Bai[1,2], Yijian Liu[1,2], Xinxuan Zhang[1,2], Xianhui Lu[1,2], Lutan Zhao[1,2], Kunpeng Wang[1,2], and Rui Hou[1,2]

[1] State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China
[2] School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China
{wangruida, baijikang, liuyijian, zhangxinxuan, luxianhui, zhaolutan, wangkunpeng, hourui}@iie.ac.cn

Abstract. FHEW/TFHE bootstrapping suffers from a structural rigidity: the accumulator's ring dimension N is limited by the input ciphertext modulus q (typically $q \leq 2N$), and thus the message space t. This coupling forces N to grow with t, leading to inflated parameters and thereby high computational costs.

In this work, we overcome this limitation by replacing the ring structure with a free $\mathcal{R}_N$-module $\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$. This generalization decouples N from q through a flexible parameter τ. We prove that computation in this extended algebra efficiently reduces to base-ring operations, enabling a new bootstrapping algorithm with improvements in both performance and precision.

Theoretically, our approach reduces the asymptotic complexity of standard FHEW/TFHE bootstrapping from quadratic $\tilde{O}(t^2)$ to quasi-linear $\tilde{O}(t)$ in the message space t. Compared with prior Extended Bootstrapping methods over discrete cyclotomic ring [PKC'23, TCC'25, Asiacrypt'25], our framework constitutes an algebraic generalization that enables more flexible parameter selection. Experimental results demonstrate that our method achieves speedups of 1.40×–2.77× over the state-of-the-art sorted extended bootstrapping [Asiacrypt'25].

Keywords: Fully Homomorphic Encryption · FHEW/TFHE · Bootstrapping

1 Introduction

Fully Homomorphic Encryption (FHE) enables arbitrary computation over encrypted data and has become a cornerstone for privacy-preserving computation. Since Gentry [27] introduced the first construction, substantial progress has been made in both efficiency and functionality [12,11,25,14,3,23,16,17,18]. For security considerations, FHE ciphertexts are encrypted with noise, which accumulates under homomorphic evaluation. *Bootstrapping*—the procedure of homomorphically evaluating the FHE decryption circuit to refresh noise—remains the only known technique that enables unlimited encrypted computation.

Early bootstrapping constructions [48,28] evaluated the decryption function via Boolean circuits. While conceptually straightforward, this approach results in very deep circuits: (a) additional cryptography assumptions are required to compress them, and (b) the resulting latency is prohibitive—up to 30 minutes to bootstrap Gentry's scheme [28].

Subsequent research focused on exploiting the structure of the decryption formula to design bootstrapping algorithms. Modern FHE schemes are mainly based on Learning with Errors (LWE) problem [44], in which decryption reduces to an inner product between the ciphertext $(\mathbf{a}, b)$ and the secret key $\mathbf{s}$, followed by modular reduction $\lfloor b - \langle \mathbf{a}, \mathbf{s} \rangle \pmod{q} \rceil$. Two main paradigms of bootstrapping have since emerged:

- **Polynomial interpolation/approximation.** Schemes such as BGV, BFV, and CKKS [12,11,25,14] realize bootstrapping by expressing modular reduction through polynomial interpolation or approximation. This approach typically incurs latency on the order of seconds. To amortize the cost, it leverages Ring-LWE (RLWE) [38] packing and batching, which in turn complicates the programming model and is most suitable for data-parallel applications.
- **Algebraic accumulator (ACC).** Another paradigm is to select an algebraic structure in which modular addition is represented by algebraic operations that can be homomorphically evaluated. This line of work has yielded dramatic reductions in latency: FHEW [23] reduced bootstrapping time to 0.1 seconds, and TFHE [16,17] brought it down to milliseconds. ACC-based bootstrapping therefore represents the most flexible and lowest-latency framework to date.

The efficiency of ACC-based bootstrapping crucially depends on the choice of the underlying algebraic structure used to encode modular operations. Over the past decade, this paradigm has evolved through several key designs:

- **AP** [3]: In 2014, Alperin and Peikert proposed the first ACC-based construction by encoding elements $i \in (\mathbb{Z}_q, +)$ into permutation matrices in the symmetric group $(\mathcal{S}_q, \times)$. Modular addition was then mapped to matrix multiplication and homomorphically evaluated via GSW inner products [29]. While this work initiated the ACC-based bootstrapping paradigm, it incurred prohibitive computational and storage costs.
- **DM-FHEW** [23]: In 2015, Ducas and Micciancio introduced polynomial rings $\mathcal{R}_N = \mathbb{Z}[X]/(X^N+1)$ as the accumulator structure, where N is a power of two. Elements $a_i \in \mathbb{Z}_q$ are encoded as $X^{a_i} \bmod (X^N + 1)$ in the multiplicative subgroup, mapping modular addition to polynomial multiplication and evaluated by ring GSW product. Combined with Fast Fourier Transform (FFT) or Number Theoretic Transform (NTT), this design achieved efficient bootstrapping and marked a milestone toward practical FHE.
- **GINX/CGGI-TFHE** [26,16]: In 2016, Gama et al. introduced efficient FHE primitives such as the external product and the controlled multiplexer (CMux) gate. Chillotti et al. subsequently incorporated these techniques into the $\mathcal{R}_N$-based bootstrapping framework, resulting in TFHE, which remains the state-of-the-art scheme for low-latency bootstrapping. This framework

was further extended to Generalized-LWE (GLWE) assumption and *programmable bootstrapping* (PBS, also known as functional bootstrapping), enabling function evaluation while simultaneous refreshing noise [9,18,19,20]. Notably, TFHE employs torus structure $\mathbb{T} = \mathbb{R}/\mathbb{Z}$ and $\mathcal{T}_N = \mathcal{R}_N/\mathbb{Z}$ to formalize message and ciphertext spaces elegantly. But in practice, $\mathbb{T}$ is discretized using $\mathbb{Z}_{q'}$ (commonly $\mathbb{Z}_{2^{32}}$ or $\mathbb{Z}_{2^{64}}$), making $\mathcal{T}_N$ computationally equivalent to $\mathcal{R}_N$. Hence, we do not distinguish $\mathcal{T}_N$ separately in this work.

Limitations of $\mathcal{R}_N$-based ACC. Recall that in the $\mathcal{R}_N$ accumulator, each element $a \in \mathbb{Z}_q$ is represented as X^a with the relation $X^{2N} = 1$. For bootstrapping correctness, the input LWE ciphertext modulus q must satisfy $q \leq 2N$, typically enforced by choosing q is a power of two and $N = q/2$.

Throughout this work, we distinguish between two moduli: the *internal modulus* q denotes the modulus of the input LWE ciphertexts to be bootstrapped, which determines the supported message precision. The *external modulus* Q corresponds to the GLWE ciphertexts encrypting the accumulator used in bootstrapping. Consequently, the coupling between q and N has several consequences:

- **Ring dimension expansion.** Since N must scale linearly with q, increasing message precision forces larger N, inflating the polynomial degree of GLWE ciphertexts and thus the size of FFT/NTTs. This increases asymptotic complexity and exacerbates implementation bottlenecks such as deeper butterfly stages, higher register pressure, and inefficient memory access.
- **External modulus blow-up.** The noise variance of bootstrapping grows as $O(nkN\sigma^2)$, where n is the LWE dimension, k is the GLWE rank, and σ is the initial noise. To maintain a negligible decryption failure rate, the external modulus Q should have size $O((\sigma q\sqrt{n})^{O(1)})$, and have quadratic growth with the message space t (see Section 5 for details).
- **Performance and precision bottleneck.** Due to the above constraints, the rapid growth of N and Q inflates both the computational complexity and the key size of bootstrapping. Moreover, in the absence of multi-precision FFT or Residue Number System (RNS)-based NTT implementations, it restricts practical libraries to only limited bootstrapping precision. For instance, OpenFHE [1], TFHEpp [39], and MOSFHET [31] support only 3-bit bootstrapping, the classical TFHE-rs implementation [4,55] achieves up to 8-bit precision, and its sparse secret key variant [5] extend to 11-bit.
- **Overhead defending IND-CPA$^{\mathbf{D}}$ attacks.** Recent cryptanalytic advances, specifically the *IND-CPA^D* security model, have demonstrated that exact FHE schemes like FHEW/TFHE are also vulnerable to key recovery attacks if the decryption failure rate (DFR) is not sufficiently negligible [36,15,13]. Defending against these attacks forces an increase in cryptosystem parameters. The structural rigidity of standard ring based method makes it sensitive to this constraint. For instance, reducing the DFR from 2^{-64} to 2^{-128} in TFHE bootstrapping incurs a $2\times$ slowdown for 6-bit message precision [6].

Existing Decoupling Attempts. To mitigate these limitations, recent works have proposed *Extended Bootstrapping (EBS)*, which corresponds to *Monic Monomial Permutation Matrix Bootstrapping* in its matrix representation [34,22]. EBS

supports high-precision bootstrapping by mapping $\mathcal{R}_N$ into a extended cyclotomic ring $\mathcal{R}_{\tau N}$ via ring homomorphism. However, a valid ring homomorphism exists only if every prime factor of τ divides the base ring degree. This constraint typically restricts τ and N to powers of two. Consequently, the resulting parameter space is discrete and sparse, preventing fine-grained optimization and leading to suboptimal performance.

These limitations raise a long-term question:

Is there an ideal algebraic structure that decouples FHE parameters more flexibly, thereby retaining the low latency of ring-based bootstrapping while scaling to higher precision and performance?

1.1 Our Results

In this paper, we introduce the Free $\mathcal{R}$-Module as a new algebraic structure for FHE. We propose our main theorem to reduce the computation over this generalized module to operations over the base ring. Based on this theory, we construct a general bootstrapping algorithm that breaks the parameter rigidity of prior ring-based methods. Our approach yields asymptotic improvements, concrete efficiency gains, and distinct resilience against IND-CPA$^{\mathrm{D}}$ attacks.

$\mathcal{R}$-Module Construction. The primary contribution of this work is the formalization of the finite free $\mathcal{R}$-module as a foundational structure for FHE:

$$\mathcal{M} = \bigoplus_{i=0}^{\tau-1} \mathcal{R} \cdot X^i,$$

where $\mathcal{R}$ is a base ring. This structure provides a novel approach to decouple algebraic capacity from the dimension of the base ring ($\mathcal{R}$'s dimension). By explicitly equipping $\mathcal{M}$ with a multiplication operation $\star$, we render it an $\mathcal{R}$-algebra. We establish a general algebraic result (Theorem 1), proving that for such an algebra structure, multiplication over $\mathcal{M}$ is reduced to Kronecker products of vectors and matrix multiplications over the base ring $\mathcal{R}$. This structural decomposition provides a fundamental guarantee for FHE design: the homomorphic evaluation of the high-dimensional module algebra strictly reduces to component-wise homomorphic operations on the underlying ring ciphertexts, see Figure 1.

Notably, this homogeneous decomposition offers distinct advantages over the natural Residue Number System (RNS). While RNS also employs product structures to decompose large rings, it necessitates arithmetic over distinct, pairwise coprime moduli. This heterogeneity imposes constraints on parameter selection and complicates hardware optimization. In contrast, our free module structure ensures that every component operates over an identical base ring.

As a paradigmatic use case, we apply this theory to upgrade FHEW/TFHE bootstrapping. This generalization resolves the rigid coupling between the ring dimension N and the LWE modulus q. For any chosen N, we select τ such that $q = 2\tau N$, establishing the module isomorphism:

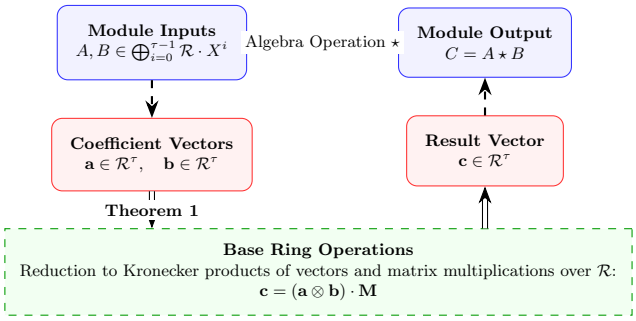


Fig. 1: Our main theorem to map module computations to base ring operations.

$$\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i \;\cong\; \mathcal{R}_{q/2}.$$

The multiplication over $\mathcal{R}_{q/2}$ induces a corresponding multiplication $\star$ over $\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$, making above isomorphism a ring isomorphism. This allows us to formulate a bootstrapping algorithm over $\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$.

Asymptotic Gain over Standard FHEW/TFHE. Building on this new algebraic structure, we achieve ACC-based bootstrapping with asymptotic and concrete improvements. These gains stem from removing the ring dimension redundancy inherent in FHEW/TFHE bootstrapping, where N must scale linearly with q (i.e., $N = \Theta(q)$) to accommodate larger message spaces—often overshooting security needs. In contrast, our approach decouples N from q. By adjusting the extension degree τ, we fix the ring dimension to strictly match the target security level, thereby removing unnecessary overhead.

- **Improved performance.** In terms of the message space t, the overall complexity reduces from quadratic growth $\tilde{O}(nt^2)$ to *quasi-linear* growth $\tilde{O}(n^{1.5}t)$, and the bootstrapping key size reduces from quadratic $\tilde{O}(nt^2)$ to *quasi-linear* $\tilde{O}(n^{1.5}t)$, where $n = \tilde{O}(\lambda)$ to be asymptotical minimal of the security level. Concretely, our method outperforms TFHE-rs bootstrapping implementation [55] by a 3.50× speedup and reduces the key size by 42.45× at its 8-bit precision limit. Compared with the sparse key TFHE variant BCLO+ at its 11-bit precision limit [5], our construction achieves a 19.61× speedup, while reducing the bootstrapping key size by a factor of 674.87×.
- **Larger precision.** The asymptotic dependence of the external modulus Q on the message space t is reduced from quadratic $\tilde{O}((\sigma\sqrt{n}t^2)^{O(1)})$ to *quasi-linear* $\tilde{O}((\sigma nt)^{O(1)})$. Moreover, for fixed t, the required internal modulus q would also be reduced from quadratic $\tilde{O}(t^2)$ to *quasi-linear* $\tilde{O}(\sqrt{n}t)$. Under the practical constraint $Q = 2^{64}$ word size, combining these two improvements raises the maximum bootstrappable message precision from the current 11-bit ceiling to beyond 32 bits.
- **Parallelism.** FHEW/TFHE bootstrapping has been regarded as inherently sequential. Even with recent key-unrolling [56,10,33,37] techniques, parallelism remains very limited, typically unlocking 2–3 threads in practice. In

Method	Structure	Ring Degree	Extended Factor	CMux
FHEW/TFHE	$\mathcal{R}_{\Phi_d(X)}$	$\phi(d) = 65536$	–	904
EBS	$\mathcal{R}_{\Phi_{\tau d}(X)}$	$\phi(d) = 2048$	$\tau = 32$	31136
SBS	$\mathcal{R}_{\Phi_{\tau d}(X)}$	$\phi(d) = 2048$	$\tau = 32$	21261
MBS (this work)	$\bigoplus_{i=0}^{\tau-1} \mathcal{R}_{\Phi_d(X)} \cdot X^i$	$\phi(d) = 2048$	$\tau = 21$	19278
Sorted MBS (this work)	$\bigoplus_{i=0}^{\tau-1} \mathcal{R}_{\Phi_d(X)} \cdot X^i$	$\phi(d) = 2048$	$\tau = 24$	13935

Table 1: Comparison with state-of-the-art schemes at 10-bit precision and 2^{-40} decryption failure rate in terms of ring degree, extended factor and CMux count.

contrast, our $\mathcal{R}_N$-module based method naturally decomposes ciphertext evaluation across τ base rings, enabling τ-way parallelism almost by design. In the ideal parallel regime, the bootstrapping complexity further approaches $O(n^2 \log n)$, which is asymptotically *quasi-constant* in the message space t.

Concrete Gain over Extended Bootstrapping. Our theoretical analysis reveals that EBS is a special case of our $\mathcal{R}$-Module bootstrapping (MBS), where the $\mathcal{R}_{\Phi_d(X)}$-module specializes to the extended ring $\mathcal{R}_{\Phi_{\tau \cdot d}(X)}$ if and only if every prime factor of τ divides d (we refer to Section 5.2 for more details). Thanks to the general structure, our work is not bound by the divisibility constraint. This allows us to select smaller, precise extended factors to match the same target bootstrapping precision, directly translating into better performance. A comparison of the extended factors and thereby the number of CMUX evaluations during bootstrapping across different methods is summarized in Table 1.

Furthermore, we integrate and generalize the algorithmic optimizations from a recent work, Sorted Extended Bootstrapping (SBS) [6], into our free $\mathcal{R}$-Module framework. Building on their insight that carefully permuting the LWE coefficients a_i reduces the required CMux evaluations, we develop an improved sorting strategy as detailed in Section 6. Experimental results demonstrate that our base MBS achieves speedups of 1.56× to 4.37× over standard extended bootstrapping. With sorting enabled, our method outperforms the state-of-the-art, achieving speedups of 1.91×–5.45× over EBS and 1.40×–2.77× over SBS.

Resilience against IND-CPA$^{\mathrm{D}}$ Attacks. Our construction neutralizes the performance penalty associated with defending against IND-CPA$^{\mathrm{D}}$ attacks. For instance, reducing DFR from 2^{-64} to 2^{-128} typically causes a 2× slowdown for standard FHEW/TFHE bootstrapping at 6-bit precision [6]. While recent extended bootstrapping methods, especially sorted bootstrapping improve resilience, they still suffer an approximately 30% performance penalty under the same setting. By contrast, our framework removes rigid parameter constraints, enabling precise parameter tuning without over-dimensioning. We explicitly derive optimized parameter sets for strict DFR targets of $\{2^{-40}, 2^{-64}, 2^{-80}, 2^{-128}\}$. Experimental results demonstrate that for 6-bit benchmark, our algorithm cuts the cost of reducing DFR from 2^{-64} to 2^{-128} to 9.4%. Furthermore, as detailed in Table 5, this advantage is systematic across all tested precisions.

1.2 Technique Overview

1.2.1 ACC Algebraic Structure

In ACC-based bootstrapping, the central task is to identify an algebraic structure in which decryption operations can be represented as algebraic computations, while allowing efficient homomorphic evaluation.

The prevailing choice is the cyclotomic ring $\mathcal{R}_N = \mathbb{Z}[X]/(X^N+1)$, where one uses the isomorphism $(\mathbb{Z}_q, +) \cong (\mathcal{H}, \times)$ with $\mathcal{H} = \langle X \rangle = \{X^a : a \in [q]\} \subseteq \mathcal{R}_{q/2}^{\times}$. This approach, however, enforces the constraint $q \leq 2N$ (typically $N = q/2$), which couples q and N, and leads to the limitations summarized above. We therefore seek a structure that decouples the polynomial degree N from q.

Attempts and Limitations. *(i) Factorization approach.* Motivated by the Chinese Remainder Theorem (CRT) decomposition method of AP bootstrapping [3], one first idea is to decompose the large polynomial modulus into smaller factors. Specifically, if $N = q/2$ is not a power of two, then $X^N + 1 = \prod_{i=0}^{\tau-1} f_i(X) \mod q$ in $\mathbb{Z}[X]$, with pairwise coprime polynomials f_i. By the Chinese Remainder Theorem,

$$\mathcal{R}_N = \mathbb{Z}[X]/(X^N+1) \;\cong\; \prod_{i=0}^{\tau-1} \mathbb{Z}[X]/(f_i(X)) =: \prod_{i=0}^{\tau-1} \mathcal{R}_i.$$

Thus computations can be carried out componentwise in smaller rings $\mathcal{R}_i$, each of degree $N_i = \deg f_i$ with $\sum_i N_i = N$, decoupling q and the per-component size N_i. Importantly, RLWE remains hard for reducible polynomials [41]. However, once N contains large odd factors, the radix-2 negacyclic structure supporting FFT/NTT disappears, rendering this approach impractical.

(ii) Block decomposition. To retain FFT/NTT efficiency, we next attempted to construct a decomposable algebraic structure isomorphic to $\mathcal{R}_{q/2}$ while keeping q a power of two. Let $N = \frac{q}{2\tau}$, any polynomial $F(X) \in \mathcal{R}_{q/2}$ can be written as

$$\begin{aligned} F(X) &= \sum_{i=0}^{N-1} a_i X^i + \Big(\sum_{i=0}^{N-1} a_{i+N} X^i\Big) X^N + \cdots + \Big(\sum_{i=0}^{N-1} a_{i+(\tau-1)N} X^i\Big) X^{(\tau-1)N} \\ &= F_0(X) + F_1(X) X^N + \cdots + F_{\tau-1}(X) X^{(\tau-1)N}, \end{aligned}$$

where each $F_j(X)$ has degree less than N. This suggests a block-wise representation, and addition indeed reduces componentwise:

$$\mathcal{R}_{q/2} \text{ (as an additive group)} \;\cong\; \bigoplus_{j=0}^{\tau-1} \mathcal{R}_N \cdot X^{jN}.$$

The obstruction arises in multiplication. Because $X^{\tau N} = -1$ in $\mathcal{R}_{q/2}$, products of terms across blocks generate *wraparound cross-terms* that cannot be localized to individual F_j. Equivalently, each $F_j(X)$ is not stably embedded into $\mathcal{R}_N$: multiplication does not preserve the block decomposition. Algebraically, the "block decomposition" provides only an additive direct sum, not a free $\mathcal{R}_N$-module. This leads to our final construction.

Our construction. The failure of the above attempt stem from the lack of a stable algebraic embedding. We then introduce two key techniques that restore algebraic compatibility and enable efficient computation.

(i) Free $\mathcal{R}_N$-module Decouples Algebraic Capacity from Ring Dimension. Instead of partitioning $F(X)$ into contiguous blocks, we then classify exponents modulo τ. Any $F(X) \in \mathcal{R}_{q/2}$ can be rewritten as

$$F(X) = \sum_{i=0}^{q/2-1} a_i X^i = \sum_{i=0}^{\tau-1} \Big(\sum_{j=0}^{N-1} a_{j\tau+i} X^{j\tau} \Big) X^i = \sum_{i=0}^{\tau-1} F_i(X^\tau) X^i,$$

where $F_i(X^\tau) = \sum_{j=0}^{N-1} a_{j\tau+i} (X^\tau)^j$. Let $Y = X^\tau$ and $N = q/(2\tau)$. Since $X^{q/2} = -1$ in $\mathcal{R}_{q/2}$, it follows that $Y^N = -1$, and thus $F_i(Y) \in \mathcal{R}_N = \mathbb{Z}[Y]/(Y^N + 1)$.

This yields a homomorphism $h : \mathcal{R}_N \to \mathcal{R}_{q/2}$ defined by $Y \mapsto X^\tau$, endowing $\mathcal{R}_{q/2}$ with an $\mathcal{R}_N$-module structure. In fact, $\mathcal{R}_{q/2}$ is a free $\mathcal{R}_N$-module with basis $\{1, X, \ldots, X^{\tau-1}\}$, giving the *module isomorphism*

$$\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i \;\cong\; \mathcal{R}_{q/2}.$$

This isomorphism allows not only additions but also a restricted class of multiplications to be reduced to additions and scalar multiplication over the module $\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$, thereby operations over $\mathcal{R}_N$. In particular, if $F(X) = F'(X^\tau)$, i.e., $F(X)$ is a polynomial whose nonzero terms all have exponents divisible by τ, then multiplying $F(X)$ by any element in $\mathcal{R}_{q/2}$ can be viewed as scalar multiplication over the module $\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$ with scalar $F'(Y)$.

Furthermore, this isomorphism between $\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$ and $\mathcal{R}_{q/2}$ allows us to define a corresponding multiplication $\star$ on the module. Specifically, if $g : \bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i \;\to\; \mathcal{R}_{q/2}$ is this isomorphism, then for any elements a, b in the module, $a \star b = g^{-1}(g(a)g(b))$. This construction confers upon the module the structure of a ring, resulting in the following ring isomorphism:

$$\left(\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i, +, \star \right) \;\cong\; (\mathcal{R}_{q/2}, +, \times).$$

Thus, while general multiplications in $\mathcal{R}_{q/2}$ can be effectively translated to the $\star$ operation on the module, the challenge remains in finding a way to express these module multiplications through operations solely within $\mathcal{R}_N$.

(ii) Reduce multiplication via $\mathcal{R}_N$-algebra. Our second step is to leverage the fact that the $\mathcal{R}_N$-module $(\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i, +, \star)$, which is isomorphic with the commutative polynomial ring $\mathcal{R}_{q/2}$, is also an $\mathcal{R}_N$-algebra.

We establish a general result: let S be a free R-module with basis $\{x_i\}_{i=0}^{\tau-1}$ that is simultaneously an R-algebra, the coefficients of the product of any two elements $a = \sum_i a_i \cdot x_i$ and $b = \sum_i b_i \cdot x_i$ (with $a_i, b_i \in R$) can be expressed as

$$(c_0, \ldots, c_{\tau-1}) = (a_0, \ldots, a_{\tau-1}) \otimes (b_0, \ldots, b_{\tau-1}) \cdot \mathbf{M}, \tag{1}$$

where $\otimes$ denotes the Kronecker product over R, and $\mathbf{M} = (m_{i,j}^{\ell}) \in R^{\tau^2 \times \tau}$ is a fixed matrix encoding the multiplication rules $x_i x_j = \sum_{\ell} m_{i,j}^{\ell} x_{\ell}$, where $i, j, \ell \in [\tau]$. This result is formalized as Theorem 1.

Applying this theorem, we conclude that *all* multiplications in $(\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i, +, \star)$ can be reduced to multiplications and additions in $\mathcal{R}_N$.

1.2.2 Homomorphic Updating of ACC.

Combining with $(\mathbb{Z}_q, +) \cong (\mathcal{H}, \times)$, where $\mathcal{H} = \langle X \rangle = \{X^a \mid a \in [q]\} \subseteq \mathcal{R}_{q/2}^{\times}$, the above algebraic development provides the mathematical foundation for constructing bootstrapping accumulators over the free module $\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$.

To encrypt a message $M \in \mathcal{R}_{q/2}$, we map M to its module representation $(M_i)_{i=0}^{\tau-1}$ and encrypt each M_i separately over $\mathcal{R}_N$. As suggested by equation (1), homomorphic multiplication over $\mathcal{R}_{q/2}$ is then reduced to efficient GGSW internal products [23] or GGSW-GLWE external products [26,16] over $\mathcal{R}_N$.

However, a general multiplication requires $O(\tau^3)$ base-ring multiplications and additions as stated in equation (1), which is impractical when τ is large. We leverage two structural optimizations make it efficient:

(i) Sparsity of the matrix M. Due to the isomorphism between $\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$ and $\mathcal{R}_{q/2}$, and the observation that in $\mathcal{R}_{q/2}$, $X^i X^j = (X^{\tau})^{\lfloor (i+j)/\tau \rfloor} X^k = Y^{\lfloor (i+j)/\tau \rfloor} \cdot X^k$ where $k = i + j - \tau\lfloor (i+j)/\tau \rfloor$, we have that for the basis X^i, X^j in the module, $X^i \star X^j = \sum_{\ell=0}^{\tau-1} m_{i,j}^{\ell} \cdot X^{\ell} = Y^{\lfloor (i+j)/\tau \rfloor} \cdot X^k$, thereby the associated multiplication matrix $\mathbf{M} = (m_{i,j}^{\ell}) \in R^{\tau^2 \times \tau}$ is sparse, containing only τ^2 nonzero entries ($m_{i,j}^{\ell} \neq 0$ iff. $\ell = i+j-\tau\lfloor (i+j)/\tau \rfloor$). This sparsity allow us to reduce the cost of $\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$ multiplication from τ^3 to τ^2 base ring operations.

(ii) Degenerate multiplications in bootstrapping. In FHEW/TFHE bootstrapping, each update of the ACC requires only two special types of multiplication: (a) between a monomial plaintext and an accumulator ciphertext, which naturally reduces to a τ-wise multiplication over the base ring; and (b) between an encrypted key and the accumulator ciphertext. In the latter case, the bootstrapping key only encrypts constant message corresponding to components of the LWE secret key s_i, whose representations in the free $\mathcal{R}_N$-module contain only a single nonzero coordinate. This property enables a refinement of the vectorized encryption by replacing redundant coordinates with zero blocks, namely:

$$\mathrm{GGSW}_{\mathsf{vec}}(M) = (\mathrm{GGSW}(M_0), \mathrm{GGSW}(0), ..., \mathrm{GGSW}(0)), \quad \text{to}$$

$$\mathrm{GGSW}_{\mathsf{vec}}(M) = (\mathrm{GGSW}(M_0), 0^{(k+1)\ell \times (k+1)}, \ldots, 0^{(k+1)\ell \times (k+1)}),$$

With this modification, the computational cost drops from τ^2 base multiplication to only τ, while noise accumulation is reduced from τ layers to 1. These optimizations ensure that our construction remains both efficient and noise-manageable even for large τ, thereby enabling practical bootstrapping over the free $\mathcal{R}_N$-module $\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$.

1.3 Organization

The remainder of this paper is organized as follows. Section 2 reviews the FHEW/TFHE foundations and present the necessary background on $\mathcal{R}$-modules and $\mathcal{R}$-algebras. Section 3 introduces our $\mathcal{R}$-Module construction. In Section 4, we present the bootstrapping algorithm over this algebraic structure. Section 5 presents the theoretical analysis and comparisons to existing approaches. Section 6 generalize our $\mathcal{R}$-Module to its sorted version. Section 7 reports the experimental results. Finally, Section 8 discuss the broader implications of our algebraic structure for other FHE topics, followed by the conclusion in Section 9.

2 Preliminary

2.1 Notations

Throughout the paper, bold letters denote vectors (or matrices). The nearest integer to r is denoted $\lfloor r \rceil$. Similarly, we denote by $\lfloor r \rceil_v := v \cdot \lfloor r/v \rceil$ the nearest multiple of v to r. The set $\mathbb{N}$ denotes the set of all positive integers. For an integer q, we identify $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$ with $\mathbb{Z} \cap [-q/2, q/2)$, and $[\cdot]_q$ denotes the $\bmod q$ reduction into $\mathbb{Z}_q$. The set $\mathbb{B}$ and $[n]$ denote $\{0,1\}$ and $\{1,2,\ldots,n\}$, respectively, for a positive integer n. For a power-of-two N, the ring $\mathbb{Z}[X]/(X^N+1)$ is denoted by $\mathcal{R}_N[X]$. We also write $\mathcal{R}_{q,N} = \mathcal{R}/q\mathcal{R} = \mathbb{Z}_q[X]/(X^N+1)$ and $\mathbb{B}_N[X] = \mathbb{B}[X]/(X^N+1)$. Unless stated otherwise, all logarithms are to the base 2.

2.2 FHEW/TFHE Cryptosystem

We briefly review the ciphertext forms and building blocks of FHEW/TFHE. We use t and q to denote the modulus of messages and ciphertexts, respectively.

LWE, RLWE, and GLWE Under a secret key $\mathbf{S} \in \mathcal{R}_{q,N}^k$, a message $M \in \mathcal{R}_{t,N}$ is encrypted into a Generalized LWE (GLWE) ciphertext $\mathbf{C} \in \mathcal{R}_{q,N}^{k+1}$ with a scaling factor Δ such that $\Delta \leq q/t$ as follows [12].

$$\mathbf{C} = \mathrm{GLWE}_{q,\mathbf{S}}(\Delta \cdot M) = (A_1, \ldots, A_k, B = \sum_{i=1}^{k} A_i \cdot S_i + [M \cdot \Delta]_q + E)$$

where $\mathbf{S} = (S_1, \ldots, S_k)$, $A_i \leftarrow \mathcal{R}_{q,N}$ for $i = 1, 2, \ldots, k$, and $E \leftarrow \chi_\sigma$ for some Gaussian distribution χ_σ. kN is called the GLWE dimension and k is called the GLWE rank. Some of subscripts $q, \mathbf{S}$ are omitted when they are clear from the context. For simplicity, B is sometimes denoted by A_{k+1}.

If $A_i = 0$ for $i = 1, 2, \ldots, k$ and $B = [\Delta \cdot M]_q$, it is called a *trivial* GLWE ciphertext. It does not provide security for the plaintext M, but only encodes M in GLWE form. A GLWE ciphertext with $N = 1$ is called an LWE ciphertext. In this case, it is common to use n to denote the LWE dimension instead of k for rank, so that an LWE ciphertext is usually denoted $(a_1, \ldots, a_n, b) \in \mathbb{Z}_q^{n+1}$. When $k = 1$, a GLWE ciphertext is called a Ring LWE (RLWE) ciphertext.

The decryption of a GLWE ciphertext is to compute its *phase*, which is defined as $B - \langle (A_1, \ldots, A_k), \mathbf{S} \rangle$, followed by rounding the phase by the scaling factor Δ. The decryption is correct if the error contained in the ciphertext is small enough to be removed by the rounding with Δ.

From the definition of the GLWE ciphertext, the sum of GLWE ciphertexts under the same secret key results in the sum of plaintexts in $\mathcal{R}_{t,N}$, with the error increasing linearly.

GLev Let $B \in \mathbb{N}$ be a power-of-two and $\ell \in \mathbb{N}$. A GLev ciphertext $\mathbf{C} \in \mathcal{R}_{q,N}^{(k+1)\ell}$ of $M \in \mathcal{R}_{q,N}$ with gadget length ℓ and base B under a GLWE secret key $\mathbf{S}$ is defined as a vector of ℓ GLWE ciphertexts of $M \in \mathcal{R}_{q,N}$ as follows.

$$\mathbf{C} = \mathrm{GLev}_{\mathbf{S}}^{(B,\ell)}(M) = \left(\mathrm{GLWE}_{\mathbf{S}}\left(v_j \cdot M\right)\right)_{j \in [\ell]}$$

where $v_j = \lceil q/B^j \rceil$ for $j = 1, \ldots, \ell$.

GGSW Let $B \in \mathbb{N}$ be a power-of-two and $\ell \in \mathbb{N}$. A GGSW ciphertext $\mathbf{C} \in \mathcal{R}_{q,N}^{(k+1)\ell \times (k+1)}$ of a message $M \in \mathcal{R}_{q,N}$ with gadget length ℓ and base B under a secret key $\mathbf{S}$ is an $(k+1)\ell \times (k+1)$ matrix over $\mathcal{R}_{q,N}$ defined as follows.

$$\mathbf{C} = \mathrm{GGSW}_{\mathbf{S}}^{(B,\ell)}(M) = \left(\mathrm{GLWE}_{\mathbf{S}}\left(v_j \cdot (-S_i \cdot M)\right)\right)_{(i,j) \in [k+1] \times [\ell]}$$

where $v_j = \lceil q/B^j \rceil$ for $j = 1, \ldots, \ell$, $\mathbf{S} = (S_1, \ldots, S_k)$, $S_{k+1} = -1$. One can also represent $\mathbf{C}$ as a vector of $k+1$ GLev ciphertexts $(\mathrm{GLev}_{\mathbf{S}}^{(B,\ell)}(-S_i \cdot M))_{i=1}^{k+1}$.

Gadget Decomposition Let $B \in \mathbb{N}$ be a power-of-two and $\ell \in \mathbb{N}$. The gadget decomposition $\mathrm{GadgetDecomp}^{(B,\ell)}$ with a base B and length ℓ decomposes an input $a \in \mathbb{Z}_q$ into a vector $(a_1, \ldots, a_\ell) \in \mathbb{Z}_q^\ell$ such that $a = \sum_{j=1}^{\ell} a_j \cdot v_j + e$, where $v_j = \lceil q/B^j \rceil$, $a_j \in \mathbb{Z} \cap [-B/2, B/2)$ for all $j = 1, \ldots, \ell$ and the decomposition error e satisfies $|e| \leq \lceil \frac{q}{2B^\ell} \rceil$. The gadget decomposition can be extended to a polynomial by applying the decomposition to its coefficients.

External Product. $(\mathrm{GGSW}(M), \mathrm{GLWE}(M')) \to \mathrm{GLWE}\ (M \cdot M')$. The external product $\boxdot$ between a GGSW ciphertext $\mathbf{C}$ and a GLWE ciphertext $\mathbf{C}'$ is defined as

$$\mathbf{C} \boxdot \mathbf{C}' = \sum_{i=1}^{k+1} \sum_{j=1}^{\ell} A_{i,j} \cdot \mathrm{GLWE}(v_j \cdot (-S_i \cdot M))$$

where $\mathrm{GLWE}(v_j \cdot (-S_i \cdot M))$ is each GLWE component of $\mathbf{C}$ for $(i,j) \in [k+1] \times [\ell]$, $(A_{i,1}, \ldots, A_{i,\ell})$ is a gadget decomposition of A_i for $i \in [k+1]$ and $\mathbf{C}' = (A_1, \ldots, A_{k+1})$, and the multiplication between a polynomial and a GLWE ciphertext denotes multiplying the polynomial to each polynomial component of the GLWE ciphertext.

CMux Gate. $(\mathrm{GGSW}(b), \mathrm{GLWE}(M^{(0)}), \mathrm{GLWE}(M^{(1)})) \to \mathrm{GLWE}(M^{(b)})$. The controlled mux gate, dubbed CMux, is the key operation used in FHEW/TFHE bootstrapping. Suppose that two GLWE ciphertexts $\mathbf{C}^{(0)}$ and $\mathbf{C}^{(1)}$ are given along with a secret boolean value b encrypted to a GGSW ciphertext $\mathbf{C}$. Then one may select $\mathbf{C}_b$ without knowing b by

$$\mathrm{CMux}(\mathbf{C}, \mathbf{C}^{(0)}, \mathbf{C}^{(1)}) = (\mathbf{C}^{(1)} - \mathbf{C}^{(0)}) \boxdot \mathbf{C} + \mathbf{C}^{(0)}.$$

2.3 R-Module and R-Algebra

Definition 1 (R-Module). *Let R be a ring with unity. A left R-module S is an abelian group $(S, +)$ equipped with a scalar multiplication map $R \times S \to S, \quad (r, s) \mapsto r \cdot s$, satisfying the following axioms for all $r, r' \in R$ and $s, s' \in S$:*

1. *Distributivity over module addition: $r \cdot (s + s') = r \cdot s + r \cdot s'$.*
2. *Distributivity over ring addition: $(r + r') \cdot s = r \cdot s + r' \cdot s$.*
3. *Associativity of scalar multiplication: $(rr') \cdot s = r \cdot (r' \cdot s)$.*
4. *Identity element action: $1_R \cdot s = s$, where 1_R is the multiplicative identity.*

A right R-module is defined analogously, with the scalar multiplication written as $m \cdot r$ and the associativity axiom adjusted to $m \cdot (rs) = (m \cdot r) \cdot s$. Unless otherwise specified, an R-module refers to a left R-module.

Definition 2 (Free R-Module). *An R-module F is called free if there exists a subset $\{x_i\}_{i \in I} \subseteq F$, called a basis, such that every element $f \in F$ can be uniquely expressed as a finite linear combination $f = \sum_{i \in I} r_i \cdot x_i$, where each $r_i \in R$ and only finitely many r_i are nonzero.*

Equivalently, F is isomorphic, as an R-module, to a direct sum of copies of R: $F \cong \bigoplus_{i \in I} R$.

Definition 3 (R-algebra). *Let R be a commutative ring with unity. An R-algebra is a ring A together with a ring homomorphism $\varphi : R \to A$ such that the image of φ lies in the center of A.*

Equivalently, A is an R-module equipped with a bilinear multiplication $A \times A \to A, \quad (a, b) \mapsto ab$, that makes A a ring with unity, and the scalar multiplication by R satisfies $r \cdot (ab) = (r \cdot a)b = a(r \cdot b)$ for all $r \in R$, $a, b \in A$.

3 Constructing the Free R-Module for ACC

Considering additive group $\mathbb{Z}_q$ and polynomial ring $\mathcal{R}_{q/2} = \mathbb{Z}[X]/(X^{q/2} + 1)$ and its multiplication subgroup $\mathcal{H} = \langle X \rangle = \{X^a | a \in [q]\} \leq \mathcal{R}_{q/2}^{\times}$, we have the following isomorphism, which is widely used in FHE bootstrapping:

$$(\mathbb{Z}_q, +) \cong (\mathcal{H}, \times)$$

Then, we show how to decouple q and N by introducing a free $\mathcal{R}_N$-module $\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$ and proving:
(a) there exists a "multiplication" $\star$ such that $\left(\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i, +, \star\right) \cong (\mathcal{R}_{q/2}, +, \times)$;
(b) $\star$ can be reduced to operations over $\mathcal{R}_N$.

3.1 High-level observation

The ring $\mathcal{R}_{q/2}$ can be viewed as the quotient of the polynomial ring $\mathbb{Z}[X]$ by the ideal generated by $X^{q/2} + 1$, i.e., polynomials over $\mathbb{Z}$ modulo $(X^{q/2} + 1)$. When

τ divides $q/2$, the exponents of X can be classified modulo τ. More precisely, for any polynomial $F(X) \in \mathcal{R}_{q/2}$, there exists a unique decomposition

$$F(X) := \sum_{i=0}^{q/2-1} a_i X^i = \sum_{i=0}^{\tau-1} \left(\sum_{j=0}^{N-1} a_{j\tau+i} X^{j\tau} \right) X^i = \sum_{i=0}^{\tau-1} F_i(X^\tau) X^i \,,$$

where $F_i(X^\tau) = \sum_{j=0}^{N-1} a_{j\tau+i} (X^\tau)^j$. Setting $Y = X^\tau$, since $X^{q/2} \equiv -1$ in $\mathcal{R}_{q/2}$, we have that $Y^{q/2\tau} = X^{q/2} = -1$, or equally, $Y^{q/2\tau} + 1 = 0$. Therefore, $F_i(Y)$ can be viewed as an element of the ring $\mathbb{Z}[Y]/(Y^{q/2\tau} + 1)$, which is isomorphic to $\mathcal{R}_{N=q/2\tau}$. This decomposition implicitly defines a group action on $\mathcal{R}_{q/2}$ under which it becomes a free $\mathcal{R}_N$-module.

To avoid confusion, in the remaining of this section, we define $\mathcal{R}_{q/2} = \mathbb{Z}[X]/(X^{q/2}+1)$, $\mathcal{R}_N = \mathbb{Z}[Y]/(Y^N+1)$. In other words, elements of the former are expressed in the form $F(X)$, while elements of the latter are expressed in the form $F(Y)$.

3.2 Free R-module

Lemma 1. *Let q be an even integer. For any positive integer τ dividing $q/2$, let $\mathcal{R}_{q/2} = \mathbb{Z}[X]/(X^{q/2}+1)$ and $\mathcal{R}_N = \mathbb{Z}[Y]/(Y^N+1)$ where $N = q/2\tau$. Then $\mathcal{R}_{q/2}$ is a free $\mathcal{R}_N$-module with basis $\{1, X, X^2, \ldots, X^{\tau-1}\}$.*

Proof (sketch). The proof consists of two facts: first, for any $N = q/2\tau$, there is a scalar multiplication map $\mathcal{R}_N \times \mathcal{R}_{q/2} \to \mathcal{R}_{q/2}$, $(r, m) \mapsto r \cdot m$ such that under this structure, $\mathcal{R}_{q/2}$ is a $\mathcal{R}_N$-module. Second, under the same scalar multiplication map, $\mathcal{R}_{q/2}$ is a free $\mathcal{R}_N$-module with basis $\{1, X, X^2, \ldots, X^{\tau-1}\}$. The whole proof is available in Appendix B.1. □

Therefore, from the above lemma, we have that:

$$\left(\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i, + \right) \cong (\mathcal{R}_{q/2}, +)$$

where "$\cong$" in the above equation means group isomorphism (in fact, a modular isomorphism), and we write such an isomorphism as $g : \bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i \to \mathcal{R}_{q/2}$. Specifically,

$$g\left(\sum_{i=0}^{\tau-1} F_i(Y) \cdot X^i \right) = \sum_{i=0}^{\tau-1} F_i(X^\tau) X^i.$$

Now, the isomorphism g and the multiplication in $\mathcal{R}_{q/2}$ could imply a multiplication $\star$ in $\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$, that is, for any $a, b \in \bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$, $a \star b = g^{-1}(g(a)g(b))$. And it follows that:

$$\left(\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i, +, \star \right) \cong (\mathcal{R}_{q/2}, +, \times)$$

However, above ring isomorphism doesn't means that the multiplication $\star$ of $\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$ can be directly reduced to the operations over $\mathcal{R}_N$. Therefore we need to introduce the concept of R-algebra.

3.3 *R*-algebra

Lemma 2. *$\left(\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i, +, \star\right)$ is a $\mathcal{R}_N$-algebra.*

Proof. Above lemma follows from the associative property of polynomial multiplication, the modular isomorphism property of g (also, g^{-1}), and the definition of "$\star$" in $\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$: For any $r \in \mathcal{R}_N$ and $a, b \in \bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$, $r \cdot (a \star b) = r \cdot g^{-1}(g(a)g(b)) = g^{-1}(r \cdot (g(a)g(b))) = g^{-1}(h(r)g(a)g(b)) = g^{-1}(g(a)(h(r)g(b))) = g^{-1}(g(a)(r \cdot g(b))) = g^{-1}(g(a)g(r \cdot b)) = a \star (r \cdot b)$. □

We now establish a fundamental theorem that characterizes the reduction of multiplications in $\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$ to operations over the base ring $\mathcal{R}_N$.

Theorem 1. *Let R be a ring and S an R-algebra that is a free R-module with basis $\{x_i\}_{i=1}^k$. Then there exists a matrix $M \in R^{k^2 \times k}$ such that for any $a = \sum_{i=1}^k a_i \cdot x_i, \quad b = \sum_{i=1}^k b_i \cdot x_i \in S$, their product satisfies $ab = \sum_{i=1}^k c_i \cdot x_i$ where $(c_1, \ldots, c_k) = (a_1, \ldots, a_k) \otimes (b_1, \ldots, b_k) \cdot M$. Here $\otimes$ denotes the vector tensor (Kronecker) product over R.*

Proof. Because S is also a free R-module, the multiplication on basis elements is given by $x_i \times x_j = \sum_{\ell=1}^k m_{i,j}^\ell x_\ell$, for some $m_{i,j}^\ell \in R$. Define a matrix $\mathbf{M} = \left(m_{i,j}^\ell\right) \in R^{k^2 \times k}$, indexed so that the row corresponds to the pair (i, j) (ordered lexicographically), and the column corresponds to ℓ.

The product of two arbitrary elements $a, b \in S$ can be computed as

$$\begin{aligned}
ab &= \left(\sum_{i=1}^k a_i \cdot x_i\right)\left(\sum_{j=1}^k b_j \cdot x_j\right) \\
&= \sum_{i=1}^k \sum_{j=1}^k (a_i b_j) \cdot (x_i x_j) && \text{(Since } S \text{ is an } R\text{-algebra)} \\
&= \sum_{i=1}^k \sum_{j=1}^k (a_i b_j) \cdot \left(\sum_{\ell=1}^k m_{i,j}^\ell x_\ell\right) \\
&= \sum_{\ell=1}^k \left(\sum_{i=1}^k \sum_{j=1}^k a_i b_j m_{i,j}^\ell\right) \cdot x_\ell.
\end{aligned}$$

Now consider the tensor (Kronecker) product of coefficient vectors: $a \otimes b := (a_1 b_1, a_1 b_2, \ldots, a_1 b_k, a_2 b_1, \ldots, a_k b_k) \in R^{k^2}$. Then the coefficients of ab relative to the basis $\{x_\ell\}$ are given by the matrix product $(c_1, \ldots, c_k) = (a \otimes b) \cdot \mathbf{M}$, where $c_\ell = \sum_{i,j} a_i b_j m_{i,j}^\ell$. This explicitly expresses the multiplication in S via the coefficients of a and b, the structure matrix $\mathbf{M}$, and the tensor product. □

3.4 Representations and Operations

Finally, we summarize representations and operations over $\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$. We use the vectorized representation $(F_0(Y), \cdots, F_{\tau-1}(Y))$ to represents the element $\sum_{i=0}^{\tau-1} F_i(Y) \cdot X^i$ in $\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$:

Representing elements of $\boldsymbol{\mathcal{R}_{q/2}}$ via $\boldsymbol{\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i}$: As established, $\mathcal{R}_{q/2}$ is a free $\mathcal{R}_N$-module with basis $\{1, X, \ldots, X^{\tau-1}\}$, which establishes the isomorphism $g : \bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i \to \mathcal{R}_{q/2}$.

Specifically, for any $F(X) \in \mathcal{R}_{q/2}$, we write it as

$$F(X) = \sum_{i=0}^{\tau-1} \left(\sum_{j=0}^{N-1} a_{ij} X^{j\tau} \right) X^i.$$

Let $F_i(Y) = \sum_{j=0}^{N-1} a_{ij} Y^j$. Then, the element of $\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$ corresponding to $F(X)$ under the isomorphism g is the vector $(F_0(Y), \ldots, F_{\tau-1}(Y))$.

Represent $\boldsymbol{\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i}$ additions over $\boldsymbol{\mathcal{R}_N}$: For any $F, F' \in \bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$ denoted by $(F_0(Y), \cdots, F_{\tau-1}(Y))$ and $(F'_0(Y), \cdots, F'_{\tau-1}(Y))$, we have that their addition $F + F'$ equals to $(F_0(Y) + F'_0(Y), \cdots, F_{\tau-1}(Y) + F'_{\tau-1}(Y))$.

Represent $\boldsymbol{\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i}$ multiplications over $\boldsymbol{\mathcal{R}_N}$: For any $F, F' \in \bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$ denoted by $(F_0(Y), \cdots, F_{\tau-1}(Y))$ and $(F'_0(Y), \cdots, F'_{\tau-1}(Y))$, we consider their product $F \star F'$.

For multiplication on basis elements $X^i \star X^j$, we write $i + j$ as

$$i + j = k + \tau \cdot \left\lfloor \frac{i+j}{\tau} \right\rfloor, \quad 0 \le k < \tau, \quad \text{then,}$$

$$X^i \star X^j = g^{-1}(g(X^i)g(X^j)) = g^{-1}(X^{i+j}) = g^{-1}((X^\tau)^{\lfloor (i+j)/\tau \rfloor} X^k) = Y^{\lfloor (i+j)/\tau \rfloor} \cdot X^k.$$

We denote by $\mathbf{M} = (m_{i,j}^\ell)$ where

$$m_{i,j}^\ell := \begin{cases} Y^{\lfloor (i+j)/\tau \rfloor}, & \text{if } \ell = i + j \mod \tau, \\ 0, & \text{otherwise.} \end{cases}$$

Then, from Theorem 1, we have that $F \star F' :=$

$$(F_0(Y), \cdots, F_{\tau-1}(Y)) \otimes (F'_0(Y), \cdots, F'_{\tau-1}(Y)) \cdot \mathbf{M},$$

which equals to

$$\left(\sum_{\substack{i,j \\ (i+j) \bmod \tau = 0}} F_i(Y) F'_j(Y)\, Y^{\lfloor (i+j)/\tau \rfloor}, \cdots, \sum_{\substack{i,j \\ (i+j) \bmod \tau = \tau - 1}} F_i(Y) F'_j(Y)\, Y^{\lfloor (i+j)/\tau \rfloor} \right).$$

4 Bootstrapping over Free $\mathcal{R}$-Module

Modern efficient bootstrapping schemes rely on an algebraic *accumulator* (ACC), which turns decryption of an LWE ciphertext into an iterative update procedure. For a ciphertext $(\mathbf{a}, b)$ under secret key $\mathbf{s}$, decryption requires

$$m = \lfloor b - \langle \mathbf{a}, \mathbf{s} \rangle \pmod{q} \rceil.$$

Bootstrapping homomorphically reproduces this process: given encrypted secret keys $E(s_i)$, one refreshes (a, b) into a fresh ciphertext $E(m)$ by evaluating the decryption circuit. ACC-based bootstrapping generally follows three phases [40]:

- Initialization of the accumulator with $E(b)$ (more generally, embedding a function f in programmable bootstrapping).
- Iterative updates by subtracting $a_i \cdot E(s_i)$, and
- Extraction of the rounded result $E(m)$ (or $E(f(m))$).

The key challenge is thus to select an algebraic structure that can encode $\mathbb{Z}_q$ into an accumulator and design the homomorphic update method. Existing FHEW/TFHE schemes are powerful but constrained by the rigid embedding $\mathbb{Z}_q \to \mathcal{R}_N$, which requires $q \leq 2N$. This tight coupling between q and N causes N to inflate with message precision and directly limits performance. Our method is to replace this rigid ring embedding with a more flexible algebraic structure: a free $\mathcal{R}_N$-module described in Section 3. This structure generalizes the $\mathcal{R}_N$-based accumulator while preserving FFT/NTT efficiency, and crucially decouples q from N.

Since the change is at the algebraic layer, *any* update strategy (FHEW-like GGSW internal product or TFHE-like GGSW-GLWE external products) can be transplanted into our framework. To make the presentation concrete, we instantiate our construction with the state-of-the-art TFHE approach.

4.1 ACC vectorized Encryption

We begin with ACC encryption. As stated in Section 3, any $M \in \bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$ admits a unique representation $(M_0(Y), \ldots, M_{\tau-1}(Y))$ with $M_i(Y) \in \mathcal{R}_N$. We then define the following vectorized encryptions by encrypting each M_i individually into a base GLWE or GGSW ciphertext.

$\mathbf{GLWE_{vec}}$ and $\mathbf{GGSW_{vec}}$

$$\mathrm{GLWE}_{\mathsf{vec}}(M) = \big(\mathrm{GLWE}(M_0), \ldots, \mathrm{GLWE}(M_{\tau-1})\big),$$

$$\mathrm{GGSW}_{\mathsf{vec}}(M) = \big(\mathrm{GGSW}(M_0), \ldots, \mathrm{GGSW}(M_{\tau-1})\big).$$

When $\tau = 1$, this collapses to the standard FHEW/TFHE representation.

4.2 Vectorized Homomorphic Operations

Having established the vectorized representation, we next define the homomorphic operations that mirror the algebraic rules of the free $\mathcal{R}_N$-module.

Plaintext–Ciphertext Multiplication. Let $M \in \bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$ be a plaintext and $\mathbf{C}' = (\mathbf{C}'_0, \cdots, \mathbf{C}'_{\tau-1}) = \mathrm{GLWE}_{\mathsf{vec}}(M')$ a ciphertext. Their multiplication is defined componentwise as

$$M \cdot \mathbf{C}' := \left(\sum_{\substack{i,j \\ (i+j) \bmod \tau = k}} (Y^{\lfloor (i+j)/\tau \rfloor} M_i) \cdot \mathbf{C}'_j \right)_{k=0}^{\tau-1}.$$

where $(Y^{\lfloor (i+j)/\tau \rfloor} M_i) \cdot \mathbf{C}'_j$ is the base plaintext-ciphertext multiplication of the plaintext $Y^{\lfloor (i+j)/\tau \rfloor} M_i$ and ciphertext $\mathbf{C}'_j$ for base GLWE encryption over $\mathcal{R}_N$.

Remark 1 (Special Case: Monomial Plaintext). If $M = X^r$ with $r = t + v\tau$, $0 \le t < \tau$, $0 \le v < N$, then

$$M_i(Y) = \begin{cases} Y^v, & i = t, \\ 0, & i \ne t, \end{cases}$$

and

$$(M \cdot \mathbf{C}')_k = \begin{cases} Y^v \cdot \mathbf{C}'_{k-t}, & k \ge t, \\ Y^{v+1} \cdot \mathbf{C}'_{k-t+\tau}, & k < t. \end{cases}$$

In other words, multiplying by a monomial plaintext in $\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i$ corresponds to a cyclic shift of the ciphertext vector $(\mathbf{C}'_0, \ldots, \mathbf{C}'_{\tau-1})$, with each wrapped component additionally multiplied by a power of Y.

Vectorized External Product. Similarly, the vectorized external product $\boxdot_{\mathsf{vec}}$ is defined as a weighted sum of componentwise external products. Let

$$\mathbf{C} = \mathrm{GGSW}_{\mathsf{vec}}(M) = \big(\mathrm{GGSW}(M_0), \ldots, \mathrm{GGSW}(M_{\tau-1})\big),$$

$$\mathbf{C}' = \mathrm{GLWE}_{\mathsf{vec}}(M') = \big(\mathrm{GLWE}(M'_0), \ldots, \mathrm{GLWE}(M'_{\tau-1})\big),$$

then

$$\mathbf{C} \boxdot_{\mathsf{vec}} \mathbf{C}' = \left(\sum_{\substack{i,j \\ (i+j) \bmod \tau = k}} Y^{\lfloor (i+j)/\tau \rfloor} \cdot (\mathbf{C}_i \boxdot \mathbf{C}'_j) \right)_{k=0}^{\tau-1}.$$

These definitions extend the classical TFHE operations to the module setting, with $\tau = 1$ recovering the standard case.

4.3 Optimization

The convolution-like rule above implies that a full vectorized external product requires τ^2 base external over $\mathcal{R}_N$, followed by τ ciphertext accumulations per component. This not only leads to quadratic computational cost in τ, but also amplifies noise growth. However, the bootstrapping degenerate structure allows a

crucial optimization. For instance, external products arise only when computing CMux gates (or inner products in FHEW-like setting), where the bootstrapping key encrypt an encoding of a component of LWE key s_i. Typically, a $\mathrm{GGSW}_{\mathsf{vec}}$ ciphertexts encrypts a constant s_i. In such setting, the message M is degenerate:

$$M = s_i X^0 \quad \rightarrow \quad M_0 = s_i,\ M_j = 0\ (j > 0).$$

That is, only the first component of the $\mathrm{GGSW}_{\mathsf{vec}}(s_i)$ ciphertext carries meaningful entropy, while the remaining $\tau - 1$ components are redundant.

We therefore modify the vectorized encryption of constant plaintext from

$$\mathrm{GGSW}_{\mathsf{vec}}(M) = (\mathrm{GGSW}(M_0), \mathrm{GGSW}(0), ..., \mathrm{GGSW}(0)), \quad \text{to}$$

$$\mathrm{GGSW}_{\mathsf{vec}}(M) = (\mathrm{GGSW}(M_0), 0^{(k+1)\ell\times(k+1)}, \ldots, 0^{(k+1)\ell\times(k+1)}),$$

where zero blocks replace unnecessary encryptions.

Under this optimization, the external product simplifies to

$$(\mathbf{C} \boxdot_{\mathsf{vec}} \mathbf{C}')_k = \mathbf{C}_0 \boxdot \mathbf{C}'_k, \qquad k = 0, \ldots, \tau - 1.$$

As a result: (a) the computational cost decreases from τ^2 base external products to only τ, and (b) noise accumulation is reduced from τ layers to a single external product. This optimization ensures that our following bootstrapping algorithm remains efficient at larger τ.

4.4 Bootstrapping Algorithm

With these building blocks, we can now instantiate bootstrapping over the free $\mathcal{R}_N$-module. The framework mirrors the standard TFHE bootstrapping pipeline.

Bootstrapping key. For an LWE secret key $s \in \{0,1\}^n$, the bootstrapping key consists of $\left\{ \mathrm{GGSW}_{\mathsf{vec}}(s_i) \right\}_{i=0}^{n-1}$, where

$$\mathrm{GGSW}_{\mathsf{vec}}(s_i) = (\mathrm{GGSW}_{S_N}(s_i), 0^{(k+1)\ell\times(k+1)}, ..., 0^{(k+1)\ell\times(k+1)}).$$

Accumulator initialization. Considering programmable bootstrapping, we encode the target function f into a trivial vectorized ciphertext:

$$\mathrm{GLWE}_{\mathsf{vec}}(X^{-b} \cdot tv) = \mathrm{GLWE}_{\mathsf{vec}}\left(X^{-b} \cdot \sum_{i=0}^{q/2-1} \Delta \cdot f\left(\left\lfloor \tfrac{it}{q} \right\rceil \right) \cdot X^i \right),$$

Update via blind rotation. Given an input ciphertext $c = (\mathbf{a}, b)$, we map each $a_i \in \mathbb{Z}_q$ into $X^{a_i} \in \mathcal{R}_{q/2}$. The blind rotation procedure aims to homomorphically update the accumulator to multiply

$$X^{-b+\sum_{i=1}^n a_i s_i} = X^{-(\Delta m + e)}$$

to the test vector. Each update is performed by a vectorized CMux gate:

$$\mathrm{CMux}_{\mathsf{vec}}(\mathbf{C}, \mathbf{C}^{(0)}, \mathbf{C}^{(1)}) = \mathbf{C} \boxdot_{\mathsf{vec}} \left(\mathbf{C}^{(1)} - \mathbf{C}^{(0)} \right) + \mathbf{C}^{(0)}.$$

Algorithm 1: Bootstrapping over Free $\mathcal{R}$-Module

Input: $\mathbf{c} = (\mathbf{a}, b) \in \mathbb{Z}_q^{n+1}$, an LWE ciphertext encrypting m under secret key $\mathbf{s}_n \in \{0,1\}^n$
Input: Bootstrapping key $\{\mathsf{BSK}_i = \mathrm{GGSW}_{\mathsf{vec}}(s_i)\}_{i=0}^{n-1}$ under base secret GGSW key $\mathbf{S}_{kN}$
Input: $\mathrm{GLWE}_{\mathsf{vec}}(tv)$, a test vector encoding function f
Input: Key switching key KSK from LWE key $\mathbf{s}_{kN}$ to $\mathbf{s}_n$
Output: $\mathbf{c}' = (\mathbf{a}', b') \in \mathbb{Z}_q^{n+1}$, a refreshed LWE ciphertext encrypting $f(m)$ under secret key $\mathbf{s}_n \in \{0,1\}^n$

$ACC \leftarrow \mathrm{GLWE}_{\mathsf{vec}}(X^{-b} \cdot tv)$ /* Initialize Accumulator */
for $i = 1$ *to* n **do**
  Let $ACC^{(0)} = ACC$
  Let $ACC^{(1)} = X^{a_i} \cdot ACC$
  $ACC \leftarrow \mathrm{CMux}_{\mathsf{vec}}(\mathsf{BSK}_i, ACC^{(0)}, ACC^{(1)})$
/* Update Accumulator via Blind Rotation */
$\mathbf{c}' \leftarrow \mathsf{SampleExtract}(ACC_0)$ /* Extract $\mathbf{c}' = \mathrm{LWE}_{kN,Q}(f(m))$ */
$\mathbf{c}' \leftarrow \mathsf{KeySwitch}(\mathbf{c}', \mathsf{KSK})$
$\mathbf{c}' \leftarrow \mathsf{ModSwitch}_{Q \to q}(\mathbf{c}')$ /* $\mathbf{c}' = \mathrm{LWE}_{n,q}(f(m))$ */
return $\mathbf{c}'$

Extraction and Key Switching. After n updates, the accumulator contains

$$\mathrm{GLWE}_{\mathsf{vec}}(X^{-(\Delta m+e)} \cdot tv).$$

We extract the constant term of ACC_0 to obtain $\mathrm{LWE}_{kN,Q}(f(m))$, and then apply key switching and modulus switching to recover $\mathrm{LWE}_{n,q}(f(m))$. These sub-algorithms are the same with the standard TFHE, we provide them in Appendix A for self completeness. The full bootstrapping algorithm is given in Algorithm 1.

4.5 Analysis

Correctness. We first propose Theorem 2 to analyze the algorithm correctness and noise growth of our bootstrapping algorithm.

Theorem 2 (Bootstrapping over $\mathcal{R}$-module). *Let $\mathbf{c}$ be an LWE ciphertext encrypting m under $\mathbf{s}_n$. The bootstrapping algorithm over $\mathcal{R}$-module (Algorithm 1) returns a refreshed LWE ciphertext as $\mathbf{c}'$ encrypted $f(m)$ with error variance*

$$\sigma_{\mathsf{pbs}}^2 = \frac{q^2}{Q^2}(\sigma_{\mathsf{br}}^2 + \sigma_{\mathsf{ks}}^2) + \sigma_{\mathsf{ms}}^2 \tag{2}$$

where $\sigma_{\mathsf{br}}^2 = n\ell_{\mathsf{br}}(k+1)N\frac{B_{\mathsf{br}}^2+2}{12}\sigma_{\mathsf{GLWE}}^2 + n\frac{q^2 - B_{\mathsf{br}}^{2\ell_{\mathsf{br}}}}{24B_{\mathsf{br}}^{2\ell_{\mathsf{br}}}}\left(1 + \frac{kN}{2}\right) + \frac{nkN}{32} + \frac{n}{16}\left(1 - \frac{kN}{2}\right)^2$,
$\sigma_{\mathsf{ks}}^2 = \ell_{\mathsf{ks}}kN\left(\frac{B_{\mathsf{ks}}^2+2}{12}\right)\sigma_{\mathsf{LWE}}^2 + kN\left(\frac{q^2 - B_{\mathsf{ks}}^{2\ell_{\mathsf{ks}}}}{24B_{\mathsf{ks}}^{2\ell_{\mathsf{ks}}}} + \frac{1}{12}\right)$, *and* $\sigma_{\mathsf{ms}}^2 = \frac{kN+4}{48} - \frac{(n+1)q^2}{12Q^2}$.

ℓ_{br}, ℓ_{ks}, and B_{br}, B_{ks} are the gadget parameters used in blind rotation and key switching. σ_{GLWE}^2 and σ_{LWE}^2 are the variances of the GLWE and LWE encryption.

The detailed step-by-step proof is provided in Appendix B.2.

Metric	AP	FHEW/TFHE	This Work
Algebraic Structure	$\Pi_{i=0}^{\tau-1}\mathcal{S}_{r_i}$	$\mathcal{R}_N$	$\bigoplus_{i=0}^{\tau-1}\mathcal{R}_N X^i$
Relation	$q=\Pi_{i=0}^{\tau-1}r_i$	$q=2N$	$q=2\tau N$
BSK Size	$(n\cdot\Sigma_{i=0}^{\tau-1}r_i)\times\ell N^2\log Q$	$n\ell(k+1)^2N\log Q$	$n\ell(k+1)^2N\log Q$
Total Complexity	$O((n\cdot\Sigma_{i=0}^{\tau-1}r_i^2+q\tau)\times N^3)$	$O(nk\ell N\log N+nk^2N)$	$O(\tau nk\ell N\log N+\tau nk^2N)$
Parallel Complexity	$O((n\cdot r_{\mathsf{max}}^2+q)\times N^3)$	$O(nk\ell N\log N+nk^2N)$	$O(nk\ell N\log N+nk^2N)$

Table 2: Metrics of our method and prior ACC-based bootstrappings. We consider the AP variant with Chinese Remainder Theorem optimization [3]

Bootstrapping key size. The bootstrapping key consists of n vectorized GGSW ciphertexts, each has only one nonzero base GGSW component, yielding a total size of $\mathrm{Size}_{\mathsf{BSK}} = n\ell(k+1)^2N\log Q$.

Computational complexity. Each vectorized CMux reduces to τ base CMux gates over $\mathcal{R}_N$. With FFT/NTT acceleration, a single base CMux requires $(k+1)\ell$ transforms, $(k+1)^2\ell$ Hadamard products, and $(k+1)$ inverse transforms. Across n iterations, the total amount is

$$n_{\mathrm{NTT}} = \tau n(k+1)(\ell+1), \qquad n_{\mathrm{HP}} = n\tau(k+1)^2\ell,$$

leading to overall complexity

$$O(\tau nk\ell N\log N+\tau nk^2N). \tag{3}$$

Remark 2 (Parallelism). Algorithm 1 can be parallelized across τ threads, with synchronization required only at line 5. Under ideal parallelism, the computational complexity reduces to$O(nk\ell N\log N+nk^2N)$.

We summarize the key metrics of AP bootstrapping, standard FHEW/TFHE bootstrapping and our $\mathcal{R}$-module bootstrapping in Table 2.

5 Comparison

5.1 Asymptotic Comparison with FHEW/TFHE

Intuitively, when compared with standard FHEW/TFHE bootstrapping, our construction compresses the bootstrapping key by replacing $n\times\mathrm{GGSW}_{q/2,Q}$ with $n\times\mathrm{GGSW}_{q/(2\tau),Q}$, and correspondingly reduces the dominant NTT costs from scaling with q to scaling with $\frac{q}{\tau}$. To detail a fair and meaningful asymptotic comparison, we fix the two fundamental invariants common to FHE schemes: (a) *concrete security*, and (b) *decryption failure rate (DFR)*. Throughout, we assume $B_{\mathrm{pbsk}}=O(B_{\mathrm{ksk}})=O(B)$ and $\ell_{\mathrm{pbsk}}=O(\ell_{\mathrm{ksk}})=O(\ell)$ so that $B^\ell=O(Q)$, and set $\sigma_{\mathsf{GLWE}}=O(\sigma_{\mathsf{LWE}})=O(\sigma)$.

5.1.1 Invariant constraints

Concrete security. The security relies on the hardness of GLWE in $\mathcal{R}_{Q,N}^k$ and LWE over $\mathbb{Z}_Q^n$. Since no known attack meaningfully exploits the algebraic structure of GLWE beyond lattice attacks, we follow the standard practice of calibrating security via LWE first and then replacing n with kN for GLWE.

Lemma 3 (Security invariant via the primal attack). *Fix a target security level as a constant, there exists an asymptotic constraint*

$$O(kN) \;\geq\; O(n) \;\geq\; O\Big(\log\Big(\tfrac{Q}{\sigma}\Big)\Big)\,. \tag{4}$$

Proof. The detailed proof is provided in Appendix B.3.

Let t be the message space (so the decryption threshold scales as $\Theta(q/t)$). Approximating the coefficient error as Gaussian, the per-coefficient failure probability can be bounded via the Gaussian tail (equivalently, erfc / erf).

Lemma 4 (DFR invariant). *Assume each coefficient error is distributed as (or upper bounded by) a centered Gaussian with variance from Theorem 2. Then for a single sub-ciphertext of ring dimension N, enforcing DFR $\leq 2^{-r}$ implies:*

$$q^2 \;\geq\; \Theta\big(t^2\, kN\;(r+\log N)\,\big),\;\; Q \;\geq\; O\left(\sigma\, t\, q\, B\cdot\sqrt{\frac{\ell nkN(r+\log N)}{q^2-t^2kN(r+\log N)}}\right). \tag{5}$$

Proof. The detailed proof is provided in Appendix B.4.

5.1.2 Metrics scaling with message space We next express the scaling of the key metrics as functions of the message space t, under the fixed invariants of Theorems 3 and 4.

Theorem 3 (Optimal t with q). *Assume security is fixed and we maximize t subject to Eq. (5). Then there exists a constant $\alpha \in (0,1)$ such that one can parameterize*

$$t \;=\; \alpha\cdot\frac{q}{\sqrt{kN\ln N}}. \tag{6}$$

- ***FHEW/TFHE (with $N=\Theta(q)$).*** *Taking $k=1$ and the standard coupling $N=q/2$, Eq. (6) implies*

$$q_{\mathsf{TFHE}} \;=\; \Theta\big(-t^2\, W_{-1}\big(-\Theta\big(\tfrac{1}{t^2}\big)\big)\big) \;=\; \Theta\big(t^2\log t\big) \qquad (t\to\infty), \tag{7}$$

  where W_{-1} is the -1 branch of the Lambert-W function.
- ***Ours (with $N=\Theta(n)$ and $\tau=\Theta(q/n)$).*** *Taking $k=1$ and $N=\Theta(n)$ (decoupled from q), Eq. (6) gives*

$$q_{\mathsf{Ours}} \;=\; \Theta\Big(t\sqrt{n\log n}\Big)\,. \tag{8}$$

Proof. The detailed proof is provided in Appendix B.5.

Corollary 1 (Asymptotic scaling of modulus, complexity, and key size). *Treat k,ℓ,r as constants and assume $B=\Theta(Q^{1/\ell})$.*

- ***External modulus*** Q. *Combining Eq. (5) with* $B = \Theta(Q^{1/\ell})$ *yields*

$$Q^{1-\frac{1}{\ell}} = O\big(\sigma q\sqrt{n}\big)\,. \tag{9}$$

Substituting Eq. (7) and Eq. (8) gives

$$Q_{\mathsf{TFHE}} = O\left(\left(\sigma\, t^2 \log t\,\sqrt{n}\right)^{\frac{\ell}{\ell-1}}\right), \qquad Q_{\mathsf{Ours}} = O\left(\left(\sigma\, t\, n\sqrt{\log n}\right)^{\frac{\ell}{\ell-1}}\right),$$

hence the asymptotic gain is $\frac{Q_{\mathsf{TFHE}}}{Q_{\mathsf{Ours}}} = \Omega\left(\left(\frac{t\log t}{\sqrt{n\log n}}\right)^{\frac{\ell}{\ell-1}}\right)$.

- ***Total complexity.*** *From Eq. (3), the total complexity can be written as*

$$O\big(nkq\log\tfrac{q}{\tau} + nk^2q\big)\,, \tag{10}$$

which is monotone in both k *and* τ*, so minimal* k *and maximal* τ *are optimal. For TFHE,* $(k,\tau) = (1,1)$ *and* $q = \Theta(t^2\log t)$ *give*

$$T_{\mathsf{TFHE}} = O\big(nt^2\log^2 t\big)\,.$$

For ours, $(k,\tau) = (1,\Theta(q/n))$ *and* $q = \Theta(t\sqrt{n\log n})$ *give*

$$T_{\mathsf{Ours}} = O\Big((n\log n)^{3/2}\, t\Big), \qquad T^{\|}_{\mathsf{Ours}} = O\big(n^2\log n\big) \quad \textit{(ideal parallelization).}$$

Thus the asymptotic gain is $\frac{T_{\mathsf{TFHE}}}{T_{\mathsf{Ours}}} = \Omega\left(\frac{t\log^2 t}{\sqrt{n\log^3 n}}\right)$.

- ***BSK size.*** *Using Eq. (9), the BSK size scales as*

$$S_{\mathsf{BSK}} = O\big(nN\cdot\log(q\sqrt{n})\big)\,.$$

For TFHE with $N = \Theta(q)$ *and* $q = \Theta(t^2\log t)$*,*

$$S_{\mathsf{BSK,TFHE}} = O\big(nt^2\log t\cdot(\log t + \log n)\big)\,.$$

For ours with $N = \Theta(n)$ *and* $q = \Theta(t\sqrt{n\log n})$*,*

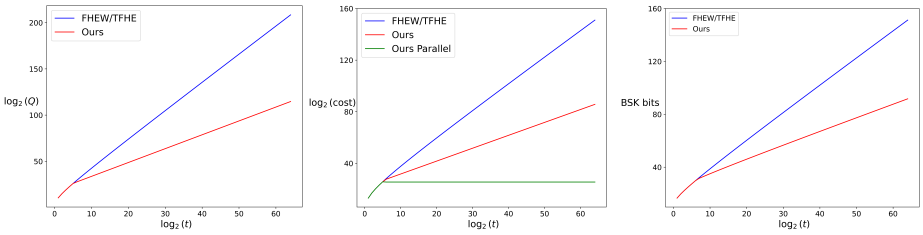
$$S_{\mathsf{BSK,Ours}} = O\Big(n^{3/2}\sqrt{\log n}\, t\cdot(\log t + \log n)\Big)\,.$$

Hence the gain is

$$\frac{S_{\mathsf{BSK,TFHE}}}{S_{\mathsf{BSK,Ours}}} = \Omega\Big(\tfrac{t\log t}{\sqrt{n\log n}}\Big)\,.$$

Proof. The detailed proof is provided in Appendix B.6.

Corollary 1 shows that, at the same message space t, our construction achieves a smaller external modulus Q, a lower total complexity, and a smaller BSK size. We summarize the asymptotic scaling with respect to t in Figure 2 and Table 3. In the small-t regime, security typically forces $q = \tilde{O}(n)$, so both methods behave similarly. As t grows, FHEW/TFHE must scale q (and thus N) accordingly, whereas our method keeps $N = \Theta(n)$ via the N–q decoupling, yielding better asymptotics.



(a) External modulus (b) Total complexity (c) BSK size

Fig. 2: Asymptotic scaling of key parameters with respect to message space t.

Metric	FHEW/TFHE	Ours	Asymptotic Gain
External modulus Q	$O\big((\sigma\sqrt{n}t^2\log t)^{\ell/(\ell-1)}\big)$	$O\big((\sigma nt\sqrt{\log n})^{\ell/(\ell-1)}\big)$	$\Omega\big((\frac{t\log t}{\sqrt{n\log n}})^{\ell/(\ell-1)}\big)$
Total complexity	$O\big(nt^2\log^2 t\big)$	$O\big(n^{1.5}t(\log n)^{1.5}\big)$	$\Omega\big(\frac{t\log^2 t}{\sqrt{n\log^3 n}}\big)$
BSK size	$O\big(nt^2\log t(\log t+\log n)\big)$	$O\big(n^{1.5}t\sqrt{\log n}(\log t+\log n)\big)$	$\Omega\big(\frac{t\log t}{\sqrt{n\log n}}\big)$

Table 3: Asymptotic scaling with message space t (treating k, ℓ as constants).

5.2 Comparison with Extended Bootstrapping Method

Extended Bootstrapping. The key insight of EBS is to enable bootstrapping over extended ring $\mathcal{R}_{\Phi_{\tau\cdot d}(X)}$ but maintaining a noise growth comparable to that of the base ring $\mathcal{R}_{\Phi_d(X)}$[3]. Specifically, by applying a homomorphism $\iota : X \mapsto X^\tau$, the GLWE/GGSW ciphertexts over the base ring are mapped to the extended ring, where they become encrypted under a discrete secret key (i.e., a key with non-zero coefficients only at powers of X^τ). This sparsity ensures that the noise growth during the external product between extended ciphertexts remains the same with that on the original ring $\mathcal{R}_{\Phi_d(X)}$, thereby achieving high-precision bootstrapping. However, EBS requires that every prime factor of the extension factor τ must divide d. This stems from their core lemma:

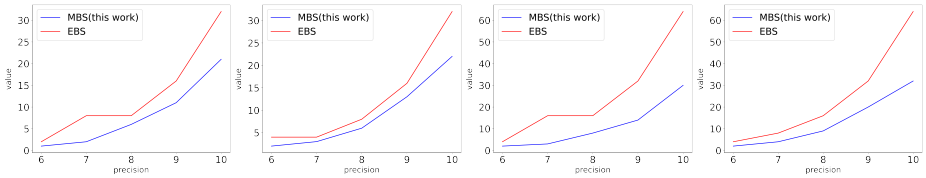
Lemma 5. *[6, Lemma 2] Let Φ_d be the d^{th} cyclotomic polynomial with degree N. Let $\tau \in \mathbb{Z}$ such that the prime factors of τ divide d. Let ι such that:*

$$\iota : \mathcal{R}_{\Phi_d(X)} \to \mathcal{R}_{\Phi_{\tau\cdot d}(X)}$$
$$P(X) = \sum_{i=0}^{N-1} a_i \cdot X^i \mapsto P_{ext}(X) = \sum_{i=0}^{N-1} a_i \cdot X^{\tau\cdot i}$$

Then, ι is a ring homomorphism.

As a counterexample, one can verify that the mapping $\iota : X \mapsto X^2$ does not induce a ring homomorphism from $\mathcal{R}_{\Phi_3(X)}$ to $\mathcal{R}_{\Phi_6(X)}$, since $\iota(\Phi_3(X)) = \iota(X^2+X+1) = X^4+X^2+1 \equiv -4X+4 \neq 0 \pmod{\Phi_6(X)}$.

[3] While originally proposed for power-of-two cyclotomics $\mathcal{R}_N$, EBS was generalized to arbitrary cyclotomic rings $\mathcal{R}_{\Phi_d(X)}$ by Bergerat et al. [6]. For theoretical rigor, we adopt this generalized notation.



(a) DFR 2^{-40} (b) DFR 2^{-64} (c) DFR 2^{-80} (d) DFR 2^{-128}

Fig. 3: Extended factor (τ) versus precision for MBS and EBS. Due to the limited set of selectable τ values in EBS, its curve exhibits a step-like behavior, whereas MBS allows arbitrary τ selection, resulting in smoother scaling.

In the meanwhile, the extended factor τ in our $\mathcal{R}$-module bootstrapping can be *any* positive integer. This allows for a continuous parameter space strictly matches the required security and noise scale. Figure 3 and Table 1 illustrated that our work can always use a smaller extended factor compared with prior works, thereby improving computational efficiency.

6 Sorted $\mathcal{R}$-Module Bootstrapping

In this section, we present the sorted $\mathcal{R}$-Module bootstrapping to further improve the concrete bootstrapping performance. We begin by briefly revisiting the sorted bootstrapping technique introduced by Bergerat et al. [6], and generalize it to the Free $\mathcal{R}$-Module setting.

6.1 Revisited Sorted Bootstrapping Algorithm

The concept of Sorted Bootstrapping (SBS) was originally proposed to reduce the redundancy in the blind rotation procedure of EBS. Bergerat et al. [6] observed that the final LWE result is extracted solely from the first component of the vecotrized accumulator (ACC_0). Consequently, updates that do not influence ACC_0 can be pruned. Specifically, the blind rotation consists of n iterations of permuting the accumulator by monomials X^{a_i} and update τ sub-accumulators by bootstrapping key. For SBS with a prime power extended factor $\tau = \mu^{\xi}$, the algorithm is divided into ξ phases, in each phase k (for $0 \leq k < \xi$) process the subset of coefficients $S_k = \{a_i \mid a_i \equiv 0 \pmod{\mu^k}$ and $a_i \not\equiv 0 \pmod{\mu^{k+1}}\}$.

- **Phase 0:** The algorithm first process coefficients in S_0 (not divisible by μ) and update the full accumulator of size τ for each $a_i \in S_0$.
- **Phase 1:** Once S_0 is exhausted, all remaining LWE coefficients are multiples of μ. The target ACC_0 now depends only on $\{ACC_0, ACC_\mu, ACC_{2\mu}, \dots\}$ during rotation by X^{a_i}. Then the algorithm discard other indices, reducing the accumulator size to τ/μ and process S_1 on this reduced accumulator.
- **Recursive Phases:** Generalizing this logic, in Phase k,the dependencies are confined to indices divisible by μ^k, restricting the active computation to a sub-accumulator of size τ/μ^k.

6.2 Generalized Sieve Mechanism for $\mathcal{R}$-Module

We now extend this sorting strategy from prime power to arbitrary composite τ, enabled for our Free $\mathcal{R}$-Module bootstrapping construction. Let $\tau = \prod_{j=1}^{k} p_j$ be the prime factorization of the extended factor, and $\pi_j = \prod_{l=1}^{j} p_l$ with $\pi_0 = 1$. We then partition the input LWE coefficients into k layers:

- **Layer 1:** Coefficients a_i such that $a_i \not\equiv 0 \pmod{p_1}$. These coefficients require the full updates of the vectorized accumulator, incurring a cost of τ CMuxs per coefficient. On average, the proportion of such coefficients is approximately $(1 - 1/p_1)$.
- **Layer j ($1 < j \leq k$):** Coefficients a_i such that $a_i \equiv 0 \pmod{\pi_{j-1}}$ but $a_i \not\equiv 0 \pmod{\pi_j}$. Since these coefficients are multiples of π_{j-1}, allowing us to perform updates in a sub-structure of size τ/π_{j-1}. The CMux cost per coefficient is reduced to τ/π_{j-1}. The probability of falling into this layer is $\frac{1}{\pi_{j-1}}(1 - \frac{1}{p_j})$.

Complexity Analysis. On average, the total expected number of CMuxs can be formulated as the sum of the expected costs of each layer:

$$N_{CMux} \approx n \sum_{j=1}^{k} \left[\underbrace{\frac{1}{\pi_{j-1}} \left(1 - \frac{1}{p_j}\right)}_{\text{Probability}} \times \underbrace{\frac{\tau}{\pi_{j-1}}}_{\text{Cost}} \right] = n \cdot \tau \sum_{j=1}^{k} \frac{1}{\pi_{j-1}^2} \left(1 - \frac{1}{p_j}\right) \tag{11}$$

Note that extended cyclotomic ring is a special kind of $\mathcal{R}$-Module, so that if we set $p_j = \mu$ for all j, Equation 11 recovers the complexity of the original SBS (Lemma 7 of [6]). Furthermore, the general $\mathcal{R}$-Module provide us two unique optimization dimensions that were absent in the sorted bootstrapping method [6], which was constrained to prime-power structures (e.g., 2^ξ):

Factor Sorting. Equation 11 reveals that the total cost depends heavily on the ordering of the factors p_j. Since the term $\frac{1}{\pi_{j-1}^2}$ decays quadratically, the cost is dominated by the early layers. More formally, we propose Lemma 6. Beyond the above intuition, the step-by-step proof is available in Appendix B.7 for self-completeness.

Lemma 6 (Smallest-Prime-First Strategy). *To minimize the bootstrapping complexity, the prime factors of τ must be processed in non-decreasing order:*

$$p_1 \leq p_2 \leq \cdots \leq p_k$$

Selection of Extended Factor τ. Our generalized complexity formula (Eq. 11) also reveals that the cost is heavily influenced by the *factorizability* of τ. Counter-intuitively, a larger composite τ with small prime factors can yield lower complexity than a smaller prime. Guided by this analysis, the extended factor τ used in our sorted $\mathcal{R}$-Module bootstrapping implementation (Section 7) are selected by minimizing the cost function in Equation 11.

6.3 General Companion Modulus Switch with Mean Compensation

The efficiency of the Generalized Sieve mechanism (Section 6.2) is bounded by the natural probability of coefficients falling into the sparse layers. To surpass this probabilistic limit, Bergerat et al. [6] introduced the *Companion Modulus Switch (CMS)*, a technique that actively forces coefficients to be multiples of a specific factor (typically $\mu = 2$). Considering the original CMS was designed for prime-power extended factors and introduces a significant rounding error, we propose the General Companion Mean Compensation Modulus Switch (GMS), which generalizes CMS to arbitrary factorizations and integrates the Mean Compensation technique [45] to mitigate the noise impact.

Our approach modifies the standard modulus switching procedure $\mathbb{Z}_Q \to \mathbb{Z}_q$ in line 8 of Algorithm 1: For up to d coefficients that do not naturally satisfy the Layer j ($1 < j \leq k$) sieving condition, we apply a forced rounding to the nearest multiple of $v = p_{j-1}$. Typically we set to $j = 2$ and $v = p_1$, the smallest prime factor of τ, this guarantees these coefficients bypass the expensive Layer 1 of the blind rotation. Since this modification amplify the rounding error, the budget d is bounded by the target decryption failure rate.

Bias Correction via Mean Compensation. LWE decryption involves the inner product between ciphertext and the LWE secret key $\mathbf{s}$. Since the secret key always follows a binary distribution (with mean $\mu_s \approx 0.5$) in TFHE setting, rounding errors accumulate into a systematic bias. We adopt the Mean Compensation technique [45] to analytically predict this amplified bias from the input a_i and μ_s, then pre-subtract it from the LWE phase b. This allows GMS to use a larger budget d than CMS without compromising security.

The formalized algorithm is detailed in Algorithm 6 in Appendix A. The average case noise analysis is formalized below, the full proof is provided in Appendix B.8.

Lemma 7 (Noise of GMS). *Let Q and q denote the input and output moduli respectively, denote $\alpha = \frac{Q}{q}$. Let d be the maximum number of companion adjustments and let v be the rounding parameter. Then the total noise introduced by the GMS algorithm can be expressed as*

$$\sigma^2_{\mathbf{gms}} = \left(\frac{1}{12} - \frac{1}{12\alpha^2}\right) + \frac{d}{4} \cdot \left(\frac{(v^3-1)\alpha^3 - 3v^2\alpha^2 + 2(v-1)\alpha}{12\alpha^2((v-1)\alpha - 1)}\right) + \frac{n-d}{4} \cdot \left(\frac{\alpha^2+2}{12\alpha^2}\right).$$

7 Implementation

In this section we implement our $\mathcal{R}$-module bootstrapping and sorted $\mathcal{R}$-module bootstrapping and compare them with prior works.

7.1 Experiments Setup.

Parameter selection. We provide the detailed parameter sets for MBS and SMBS in Appendix C (see Tables 7 and 8, respectively). Parameter precision can

Scheme	Metric	$t = 2^4$	$t = 2^5$	$t = 2^6$	$t = 2^7$	$t = 2^8$	$t = 2^9$	$t = 2^{10}$	$t = 2^{11}$
SMBS (Ours)	Time (ms)	9.5307	12.093	14.015	20.296	48.002	90.522	276.350	480.570
	Key Size (MB)	46.96	44.17	51.18	51.64	51.64	55.05	120.32	134.21
PBS	Time (ms)	10.121	12.684	31.538	82.512	168.09	–	–	–
		×1.06	×1.05	×2.25	×4.06	×3.50	–	–	–
	Key Size (MB)	49.8	98.5	231.3	448.0	2192.0	–	–	–
		×1.06	×2.23	×4.52	×8.67	×42.45	–	–	–
BCLO+ [5]	Time (ms)	8.6126	11.844	38.712	87.928	250.87	627.64	1912.1	9427.4
		×0.90	×0.98	×2.76	×4.33	×5.22	×6.93	×6.92	×19.61
	Key Size (MB)	55.4	52.5	210.0	448.0	1452.0	4096.0	13152.0	90560.0
		×1.18	×1.19	×4.10	×8.67	×28.12	×74.42	×109.35	×674.87

Table 4: Comparison of Sorted MBS execution time and bootstrapping key size with TFHE bootstrapping across message spaces ranging from 2^4 to 2^{11}, where all parameter sets achieve a decryption failure rate below 2^{-40}.

be up to 32-bit. These parameters are calibrated to achieve 128-bit security while satisfying strict decryption failure rates of 2^{-40}, 2^{-64}, 2^{-80}, and 2^{-128}. Security estimates are derived using the Lattice Estimator[4] with `red_cost_model=MATZOV`. Full experimental results for bootstrapping precisions up to 20-bit are listed in Table 10 of Appendix D. For the comparisons in the following sections, we report only the subset of results that aligns with the precisions of the baseline schemes. Furthermore, to demonstrate the high-precision scalability of MBS, we provide a parameter instantiation for 32-bit bootstrapping in Table 7.

Environment. Benchmarks were run on a server comprising AMD EPYC 9474F with two 48-Core Processor @ 3.6 GHz and 1.5 TB memory.

For a fair comparison with prior works, all experiments are restricted to a single thread, and the reported results are averaged over more than 100 runs.

7.2 Comparison with TFHE Bootstrapping

We compare our construction with TFHE programmable bootstrapping as implemented in TFHE-rs [55] and the sparse-key BCLO+ variant [5]. To ensure a fair comparison, all schemes are tested within the TFHE-rs library, targeting a unified DFR of 2^{-40}. The performance results are summarized in Table 4.

For small message spaces $t \leq 2^5$, both our scheme and prior ACC-based constructions should select $kN \geq q$ to meet the security requirement. In this regime, our extended factor defaults to $\tau = 1$. Consequently, our construction degenerates to the standard TFHE. Under these conditions, BCLO+ exhibits better efficiency due to its specialized partial key optimization.

A critical performance divergence occurs when the message space exceeds 2^5. Beyond this threshold, the parameter setting $(N = 2048, k = 1)$ for GLWE ciphertexts is no longer enough for LWE ciphertext $q = 8192$. Standard TFHE

[4] https://github.com/malb/lattice-estimator
Commit ID: `5ba00f56dd1086c3a42b98fc596c64907adb96ff`

and BCLO+ must increase the polynomial degree N to cover q, whereas our method instead increases the extended factor τ. Compared with classical programmable bootstrapping, this structural advantage translates into a consistent speedup of **2.25**×–**3.50**×, and the BSK size is reduced by **4.52**×–**42.45**×. The performance gap widens as the message space grows. Furthermore, our method also achieves a **2.76**×–**19.61**× speedup Against BCLO+. At a message space of 2^{11}, BCLO+ approaches its parameter limits and requires a large gadget decomposition parameter ($\ell = 20$) to control bootstrapping noise, whereas our scheme continues to scale with $\ell = 2$, creating a substantial performance gap. In terms of BSK size, we observe a **4.1**×–**674.87**× reduction. This trend again follows from increasing τ rather than N, which avoids the BSK growth seen in BCLO+.

7.3 Comparison with Extended Bootstrapping Methods

We benchmark our free $\mathcal{R}$-Module bootstrapping (MBS) and its sorted variant (SMBS) against the extended bootstrapping method (EBS) [34,22] and the sorted bootstrapping (SBS) [6]. As established in Section 5.2, extended bootstrapping methods are theoretically special cases of our framework. Consequentially, to ensure a fair comparison, we implement EBS and SBS within our codebase, using the exact parameters specified in the recent papers[5] [6]. These implementations serve as our primary baselines. The detailed comparison is summarized in Table 5.

Compared to bootstrapping over extended cyclotomic rings, our Free $\mathcal{R}$-module bootstrappings offer greater parameter flexibility and noise management. This structural advantage translates into better performance: MBS outperforms EBS by **1.56**×–**4.37**×, and SMBS outperforms SBS by **1.40**×–**2.77**×. Furthermore, the internal performance gain of SMBS over MBS (approx. **1.0**×–**1.45**×) stems specifically from the algorithmic optimizations introduced by our generalized sorting strategy and the GMS algorithm. When the precision increases from 2^9 to 2^{10}, both the execution time and the key size grow significantly faster than in previous steps. This is because l_{pbs} increases from 1 to 2 at this point. Beyond the scaling caused by the increase in τ, this change introduces an additional twofold overhead, resulting in the apparent jump observed in the measurements.

7.4 Comparison with Tree-Based PBS

We benchmark our free $\mathcal{R}$-Module bootstrapping (MBS) against the tree-based programmable bootstrapping constructions [30,46]. These methods operate by partitioning high-precision messages into blocks and executing bootstrapping

[5] We identified an open-source implementation provided by the authors of [6] at https://github.com/zama-ai/tfhe-rs/tree/artifact_asiacrypt_2025. For completeness, we evaluated this artifact and report the results in Table 9 of Appendix D. As detailed in the table, their code runs 10%–22% faster than our baselines. However, even accounting for this implementation gap, our MBS and SMBS schemes remain strictly faster, demonstrating the dominance of our algorithmic improvements.

Scheme	DFR	$t = 2^6$	$t = 2^7$	$t = 2^8$	$t = 2^9$	$t = 2^{10}$
SMBS (This Work)	2^{-40}	14.015 (×1.00)	20.296 (×1.00)	48.002 (×1.00)	90.522 (×1.00)	276.350 (×1.00)
	2^{-64}	17.888 (×1.00)	28.004 (×1.00)	48.955 (×1.00)	123.310 (×1.00)	320.580 (×1.00)
	2^{-80}	18.039 (×1.00)	28.603 (×1.00)	61.601 (×1.00)	122.310 (×1.00)	398.310 (×1.00)
	2^{-128}	19.745 (×1.00)	32.698 (×1.00)	76.399 (×1.00)	166.730 (×1.00)	417.420 (×1.00)
MBS (This Work)	2^{-40}	13.946 (×1.00)	25.625 (×1.26)	66.996 (×1.40)	126.269 (×1.39)	369.826 (×1.34)
	2^{-64}	23.702 (×1.33)	34.302 (×1.22)	68.061 (×1.39)	161.941 (×1.31)	413.636 (×1.29)
	2^{-80}	23.801 (×1.32)	35.660 (×1.25)	90.752 (×1.47)	175.099 (×1.43)	566.506 (×1.42)
	2^{-128}	24.329 (×1.23)	47.146 (×1.44)	105.625 (×1.38)	240.773 (×1.44)	602.314 (×1.44)
SBS[6]	2^{-40}	22.252 (×1.59)	53.340 (×2.63)	95.689 (×1.99)	191.206 (×2.11)	407.113 (×1.47)
	2^{-64}	31.191 (×1.74)	56.458 (×2.02)	107.061 (×2.19)	216.598 (×1.76)	449.839 (×1.40)
	2^{-80}	33.658 (×1.87)	75.352 (×2.63)	149.475 (×2.43)	307.290 (×2.51)	641.520 (×1.61)
	2^{-128}	41.093 (×2.08)	90.580 (×2.77)	179.797 (×2.35)	374.519 (×2.25)	794.830 (×1.90)
EBS[34]	2^{-40}	26.725 (×1.91)	75.513 (×3.72)	146.352 (×3.05)	294.699 (×3.26)	604.757 (×2.19)
	2^{-64}	41.092 (×2.30)	75.864 (×2.71)	152.166 (×3.11)	309.411 (×2.51)	645.960 (×2.01)
	2^{-80}	50.093 (×2.78)	155.770 (×5.45)	282.680 (×4.59)	582.647 (×4.76)	1200.473 (×3.01)
	2^{-128}	54.233 (×2.75)	144.130 (×4.41)	290.206 (×3.80)	596.175 (×3.58)	1236.148 (×2.96)

Table 5: Execution time comparison among MBS, SMBS, EBS, and SBS across precisions 6–10 and decryption failure rates 2^{-40}, 2^{-64}, 2^{-80}, and 2^{-128}.

Scheme	$t = 2^8$	$t = 2^{12}$
MBS (Ours)	126.27ms (×1.00)	3.41s (×1.00)
Tree Based	164.15ms (×1.30)	5.98s (×1.75)

Table 6: Comparison between MBS and tree-based PBS at precisions 8 and 12. The original tree-based parameters target a 2^{-23} DFR, while MBS target 2^{-40}.

within a recursive tree structure. To ensure a fair comparison for arbitrary Look-Up Table (LUT) evaluation, we configure MBS to reserve one bit of message space for the negacyclic property, contrasting with the standard 4-bit block decomposition used in the baseline [46]. While both approaches exhibit exponential scaling with precision, the tree-based method incurs specific overheads, notably the computationally expensive "Repacking" step required to bridge layers. As shown in Table 6, by avoiding this repacking bottleneck, MBS outperforms the baseline at precisions of 2^8 and 2^{12} (achieving **1.30**× and **1.75**× speedups, respectively), even while targeting a significantly stricter DFR (2^{-40} vs. 2^{-23}). We acknowledge, however, that the tree-based approach remains a potentially better choice for multivariate homomorphic evaluations.

8 Broader Implications of the $\mathcal{R}_N$-Module Structure

In addition to a concrete bootstrapping algorithm, the core contribution of this paper is the introduction of the $\mathcal{R}_N$-module algebraic structure. By decoupling parameters that were previously rigidly tied, this framework provides a powerful and versatile foundation for re-examining a range of FHE techniques. This section highlights its implications for several central topics in FHE.

Automorphism-Based Bootstrapping and Circuit Bootstrapping. A promising direction in recent research is bootstrapping via homomorphic automorphisms [35,52,53,37,7]. However, existing constructions still suffer from the

rigid coupling between N and q. Our algebraic framework provides a natural way to break this dependency: the isomorphism

$$\left(\bigoplus_{i=0}^{\tau-1} \mathcal{R}_N \cdot X^i, +, \star\right) \cong (\mathcal{R}_{q/2}, +, \times),$$

We then claim that it also enables automorphisms on $\mathcal{R}_{q/2}$ to be efficiently decomposed into automorphisms on the smaller ring $\mathcal{R}_N$:

Theorem 4 (Automorphism). *Let $\sigma_a : \mathcal{R}_{q/2} \to \mathcal{R}_{q/2}$ be an automorphism of the large ring defined by*

$$\sigma_a(X) = X^a, \qquad \gcd(a, q) = 1.$$

Suppose $F(X) \in \mathcal{R}_{q/2}$ with module representation $(F_0(Y), \cdots, F_{\tau-1}(Y))$, then $\sigma_a(F(X))$ has module representation that:

$$\left(Y^{\left\lfloor \frac{ak_0}{\tau} \right\rceil} \psi_a(F_{k_0}), \cdots, Y^{\left\lfloor \frac{ak_{\tau-1}}{\tau} \right\rceil} \psi_a(F_{k_{\tau-1}})\right),$$

where for each $0 \le j \le \tau - 1$, $k_j \equiv a^{-1} \cdot j \pmod{\tau}$, and $\psi_a : \mathcal{R}_N \to \mathcal{R}_N$ is the automorphism of defined by $\psi_a(Y) = Y^a$.

The proof is provided in Appendix B.9 for self completeness. This property also directly benefits circuit bootstrapping [17,51,52,50,49], where homomorphic automorphisms and thereby homomorphic trace evaluation are critical building blocks for converting redundant GLWE ciphertexts into GGSW ciphertexts.

Integration with PBSManyLUT Our $\mathcal{R}_N$-module bootstrapping is compatible with a broad class of programmable bootstrapping techniques, including full domain bootstrapping constructions and multi-output extensions of PBS. The more interesting case is PBSmanyLUT [20], beyond parallel multi-output evaluation, it brings an additional benefit *inside our bootstrapping algorithm*. As in PBSmanyLUT clears ϑ noise positions in the bootstrapped ciphertext, it can reduce the MBS latency by approximately a factor of 2^{ϑ}. This insight—exploiting coefficient divisibility to prune the computation graph—mirrors the core principle underlying the Companion Modulus Switch (CMS) detailed in Section 6.

NTRU Bootstrapping. NTRU-based FHE [54,8,53,37] has been explored as a potentially more efficient alternative to LWE/RLWE-based schemes. However, NTRU exhibits a distinctive limitation: when the ciphertext modulus Q exceeds the fatigue point $n^{2.484+o(1)}$ (under the condition $\log_n(\sigma) = o(1)$), sublattice attacks lead to a sharp degradation in concrete security [24]. This imposes a stringent upper bound on external modulus setting in NTRU bootstrapping, which restricts the scalability of NTRU bootstrapping. Our proposed $\mathcal{R}_N$-module framework slows down the growth of Q with respect to q. As a result, it becomes possible to select larger q (thereby larger message space t) while still keeping Q below the fatigue threshold. This potentially enables efficient NTRU bootstrapping at higher precision.

Parallelism and Hardware Acceleration As theoretically established in Section 5, our proposed Free $\mathcal{R}$-Module framework inherently exposes fine-grained parallelism, reducing the asymptotic complexity to quasi-constant time in the ideal parallel regime. Recent work on sorted bootstrapping has successfully explored parallel execution strategies on general-purpose CPUs [6], we then extend this trajectory by exploring the potential of our construction on hardware accelerators. We refer to Appendix E for an analysis of the hardware implications.

9 Conclusion

Bootstrapping remains the principal bottleneck for advancing the efficiency and scalability of fully homomorphic encryption. In this work, we revisit the algebraic foundations of accumulator-based bootstrapping and address the long-standing rigidity of ring-centric designs, where the ciphertext modulus and polynomial dimension are tightly coupled. By introducing a free $\mathcal{R}_N$-module structure, we establish a more general algebraic framework that not only subsumes the classical $\mathcal{R}_N$-based construction but, more importantly, decouples the modulus q from the dimension N. This flexibility yields improvements in attainable precision, bootstrapping complexity, and key size, while simultaneously enabling scalable parallelization and robustness against IND-CPA$^{\mathrm{D}}$ attacks. Beyond these, the proposed $\mathcal{R}$-module framework also serves as a unifying perspective for re-examining and potentially enhancing a broader range of homomorphic techniques.

References

1. Al Badawi, A., Bates, J., Bergamaschi, F., Cousins, D.B., Erabelli, S., Genise, N., Halevi, S., Hunt, H., Kim, A., Lee, Y., Liu, Z., Micciancio, D., Quah, I., Polyakov, Y., R.V., S., Rohloff, K., Saylor, J., Suponitsky, D., Triplett, M., Vaikuntanathan, V., Zucca, V.: Openfhe: Open-source fully homomorphic encryption library. In: Proceedings of the 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography. pp. 53–63. WAHC'22, Association for Computing Machinery, New York, NY, USA (2022). https://doi.org/10.1145/3560827.3563379, https://doi.org/10.1145/3560827.3563379
2. Alkim, E., Ducas, L., Pöppelmann, T., Schwabe, P.: Post-quantum key {Exchange—A} new hope. In: 25th USENIX Security Symposium (USENIX Security 16). pp. 327–343 (2016)
3. Alperin-Sheriff, J., Peikert, C.: Faster bootstrapping with polynomial error. In: Annual Cryptology Conference. pp. 297–314. Springer (2014)
4. Bergerat, L., Boudi, A., Bourgerie, Q., Chillotti, I., Ligier, D., Orfila, J.B., Tap, S.: Parameter Optimization and Larger Precision for (T)FHE. Journal of Cryptology **36**, 28 (2023). https://doi.org/10.1007/s00145-023-09463-5
5. Bergerat, L., Chillotti, I., Ligier, D., Orfila, J.B., Roux-Langlois, A., Tap, S.: New secret keys for enhanced performance in (t)fhe. In: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. p. 2547–2561. CCS '24, Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3658644.3670376, https://doi.org/10.1145/3658644.3670376
6. Bergerat, L., Orfila, J.B., Roux-Langlois, A., Tap, S.: Accelerating tfhe with sorted bootstrapping techniques. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 68–100. Springer (2025)
7. Bernard, O., Joye, M.: Bootstrapping (t) fhe ciphertexts via automorphisms: Closing the gap between binary and gaussian keys. Cryptology ePrint Archive (2025)
8. Bonte, C., Iliashenko, I., Park, J., Pereira, H.V.L., Smart, N.P.: FINAL: Faster FHE Instantiated with NTRU and LWE. In: Agrawal, S., Lin, D. (eds.) ASIACRYPT 2022. pp. 188–215. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-22966-4_7
9. Boura, C., Gama, N., Georgieva, M., Jetchev, D.: Simulating Homomorphic Evaluation of Deep Learning Predictions. In: Dolev, S., Hendler, D., Lodha, S., Yung, M. (eds.) Cyber Security Cryptography and Machine Learning. vol. 11527, pp. 212–230. Springer (2019)
10. Bourse, F., Minelli, M., Minihold, M., Paillier, P.: Fast homomorphic evaluation of deep discretized neural networks. In: Annual International Cryptology Conference. pp. 483–512. Springer (2018)
11. Brakerski, Z.: Fully Homomorphic Encryption without Modulus Switching from Classical GapSVP. In: Safavi-Naini, R., Canetti, R. (eds.) CRYPTO 2012. vol. 7417, pp. 868–886. Springer (2012). https://doi.org/10.1007/978-3-642-32009-5_50
12. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (Leveled) Fully Homomorphic Encryption without Bootstrapping. In: Proceedings of the 3rd Innovations in Theoretical Computer Science Conference. p. 309–325. ACM (2012). https://doi.org/10.1145/2633600
13. Checri, M., Sirdey, R., Boudguiga, A., Bultel, J.P.: On the practical cpa d security of "exact" and threshold fhe schemes and libraries. In: Annual International Cryptology Conference. pp. 3–33. Springer (2024)

14. Cheon, J.H., Kim, D., Kim, Y., Song, Y.: Ensemble Method for Privacy-Preserving Logistic Regression Based on Homomorphic Encryption. IEEE Access **6**, 46938–46948 (2018)
15. Cheon, J.H., Choe, H., Passelègue, A., Stehlé, D., Suvanto, E.: Attacks against the ind-cpad security of exact fhe schemes. In: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. pp. 2505–2519 (2024)
16. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Faster Fully Homomorphic Encryption: Bootstrapping in Less Than 0.1 Seconds. In: Cheon, J.H., Takagi, T. (eds.) Advances in Cryptology – ASIACRYPT 2016. vol. 10031, pp. 3–33. Springer (2016)
17. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Faster Packed Homomorphic Operations and Efficient Circuit Bootstrapping for TFHE. In: Takagi, T., Peyrin, T. (eds.) Advances in Cryptology – ASIACRYPT 2017. pp. 377–408. Springer International Publishing, Cham (2017)
18. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: TFHE: Fast Fully Homomorphic Encryption Over the Torus. Journal of Cryptology **33**, 34–91 (2020). https://doi.org/10.1007/s00145-019-09319-x
19. Chillotti, I., Joye, M., Paillier, P.: Programmable Bootstrapping Enables Efficient Homomorphic Inference of Deep Neural Networks. In: Dolev, S., Margalit, O., Pinkas, B., Schwarzmann, A. (eds.) Cyber Security Cryptography and Machine Learning. pp. 1–19. Springer International Publishing, Cham (2021). https://doi.org/10.1007/978-3-030-78086-9_1
20. Chillotti, I., Ligier, D., Orfila, J.B., Tap, S.: Improved Programmable Bootstrapping with Larger Precision and Efficient Arithmetic Circuits for TFHE. In: Tibouchi, M., Wang, H. (eds.) ASIACRYPT 2021. pp. 670–699. Springer (2021). https://doi.org/10.1007/978-3-030-92078-4_23
21. Deng, X., Fan, S., Hu, Z., Tian, Z., Yang, Z., Yu, J., Cao, D., Meng, D., Hou, R., Li, M., Lou, Q., Zhang, M.: Trinity: A general purpose fhe accelerator. In: 2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO). pp. 338–351 (2024). https://doi.org/10.1109/MICRO61859.2024.00033
22. Duan, K., Li, H., Liu, D., Ma, G.: Large-plaintext functional bootstrapping in fhe with small bootstrapping keys. In: Theory of Cryptography Conference. pp. 253–279. Springer (2025)
23. Ducas, L., Micciancio, D.: FHEW: Bootstrapping Homomorphic Encryption in Less Than a Second. In: Oswald, E., Fischlin, M. (eds.) Advances in Cryptology – EUROCRYPT 2015. vol. 9056, pp. 617–640. Springer (2015)
24. Ducas, L., van Woerden, W.: NTRU Fatigue: How Stretched is Overstretched? In: Tibouchi, M., Wang, H. (eds.) Advances in Cryptology – ASIACRYPT 2021. pp. 3–32. Springer International Publishing, Cham (2021)
25. Fan, J., Vercauteren, F.: Somewhat Practical Fully Homomorphic Encryption. IACR Cryptology ePrint Archive, Report 2012/144 (2012), https://eprint.iacr.org/2012/144
26. Gama, N., Izabachène, M., Nguyen, P.Q., Xie, X.: Structural lattice reduction: generalized worst-case to average-case reductions and homomorphic cryptosystems. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 528–558. Springer (2016)
27. Gentry, C.: Fully Homomorphic Encryption Using Ideal Lattices. In: Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing. p. 169–178. ACM (2009)

28. Gentry, C., Halevi, S.: Implementing gentry's fully-homomorphic encryption scheme. In: Annual international conference on the theory and applications of cryptographic techniques. pp. 129–148. Springer (2011)
29. Gentry, C., Sahai, A., Waters, B.: Homomorphic Encryption from Learning with Errors: Conceptually-Simpler, Asymptotically-Faster, Attribute-Based. In: Canetti, R., Garay, J.A. (eds.) CRYPTO 2013. vol. 8042, pp. 75–92. Springer (2013). https://doi.org/10.1007/978-3-642-40041-4_5
30. Guimarães, A., Borin, E., Aranha, D.F.: Revisiting the functional bootstrap in tfhe. IACR Transactions on Cryptographic Hardware and Embedded Systems pp. 229–253 (2021)
31. Guimarães, A., Borin, E., Aranha, D.F.: MOSFHET: Optimized Software for FHE over the Torus. Journal of Cryptographic Engineering **14**(3), 577–593 (Jul 2024). https://doi.org/10.1007/s13389-024-00359-z, https://doi.org/10.1007/s13389-024-00359-z
32. Jiang, L., Lou, Q., Joshi, N.: Matcha: a fast and energy-efficient accelerator for fully homomorphic encryption over the torus. p. 235–240. DAC '22, Association for Computing Machinery, New York, NY, USA (2022). https://doi.org/10.1145/3489517.3530435, https://doi.org/10.1145/3489517.3530435
33. Joye, M., Paillier, P.: Blind rotation in fully homomorphic encryption with extended keys. In: International Symposium on Cyber Security, Cryptology, and Machine Learning. pp. 1–18. Springer (2022)
34. Lee, K.H., Yoon, J.W.: Discretization error reduction for high precision torus fully homomorphic encryption. In: IACR International Conference on Public-Key Cryptography. pp. 33–62. Springer (2023)
35. Lee, Y., Micciancio, D., Kim, A., Choi, R., Deryabin, M., Eom, J., Yoo, D.: Efficient FHEW Bootstrapping with Small Evaluation Keys, and Applications to Threshold Homomorphic Encryption. In: Hazay, C., Stam, M. (eds.) Advances in Cryptology – EUROCRYPT 2023. pp. 227–256. Springer Nature Switzerland, Cham (2023)
36. Li, B., Micciancio, D.: On the security of homomorphic encryption on approximate numbers. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 648–677. Springer (2021)
37. Li, Z., Lu, X., Wang, Z., Wang, R., Liu, Y., Zheng, Y., Zhao, L., Wang, K., Hou, R.: Faster ntru-based bootstrapping in less than 4 ms. IACR Transactions on Cryptographic Hardware and Embedded Systems **2024**(3), 418–451 (2024)
38. Lyubashevsky, V., Peikert, C., Regev, O.: On ideal lattices and learning with errors over rings. In: Annual international conference on the theory and applications of cryptographic techniques. pp. 1–23. Springer (2010)
39. Matsuoka, K.: TFHEpp: pure C++ implementation of TFHE cryptosystem. https://github.com/virtualsecureplatform/TFHEpp (2020)
40. Micciancio, D., Polyakov, Y.: Bootstrapping in fhew-like cryptosystems. In: Proceedings of the 9th on Workshop on Encrypted Computing & Applied Homomorphic Cryptography. pp. 17–28 (2021)
41. Peikert, C., Regev, O., Stephens-Davidowitz, N.: Pseudorandomness of ring-lwe for any ring and modulus. In: Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing. pp. 461–473 (2017)
42. Prasetiyo, Putra, A., Kim, J.Y.: Morphling: A throughput-maximized tfhe-based accelerator using transform-domain reuse. In: 2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA). pp. 249–262 (2024). https://doi.org/10.1109/HPCA57654.2024.00028

43. Putra, A., Prasetiyo, Chen, Y., Kim, J., Kim, J.Y.: Strix: An end-to-end streaming architecture with two-level ciphertext batching for fully homomorphic encryption with programmable bootstrapping. In: Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture. p. 1319–1331. MICRO '23, Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3613424.3614264, https://doi.org/10.1145/3613424.3614264
44. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. Journal of the ACM (JACM) **56**(6), 1–40 (2009)
45. de Ruijter, T., D'Anvers, J.P., Verbauwhede, I.: Don't be mean: Reducing approximation noise in tfhe through mean compensation. Cryptology ePrint Archive (2025)
46. Trama, D., Boudguiga, A., Clet, P.E., Sirdey, R., Ye, N.: Designing a general-purpose 8-bit (t) fhe processor abstraction. IACR Transactions on Cryptographic Hardware and Embedded Systems **2025**(2), 535–578 (2025)
47. Van Beirendonck, M., D'Anvers, J.P., Turan, F., Verbauwhede, I.: Fpt: A fixed-point accelerator for torus fully homomorphic encryption. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. p. 741–755. CCS '23, Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3576915.3623159, https://doi.org/10.1145/3576915.3623159
48. Van Dijk, M., Gentry, C., Halevi, S., Vaikuntanathan, V.: Fully homomorphic encryption over the integers. In: Annual international conference on the theory and applications of cryptographic techniques. pp. 24–43. Springer (2010)
49. Wang, R., Bai, J., Shen, X., Lu, X., Li, Z., Xiang, B., Wang, Z., Wang, H., Zhao, L., Wang, K., et al.: Tetris: Versatile tfhe lut and its application to fhe instruction set architecture. Cryptology ePrint Archive (2025)
50. Wang, R., Ha, J., Shen, X., Lu, X., Chen, C., Wang, K., Lee, J.: Refined tfhe leveled homomorphic evaluation and its application. In: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security. CCS '25, ACM (2025)
51. Wang, R., Wei, B., Li, Z., Lu, X., Wang, K.: Tfhe bootstrapping: Faster, smaller and time-space trade-offs. In: Australasian Conference on Information Security and Privacy. pp. 196–216. Springer (2024)
52. Wang, R., Wen, Y., Li, Z., Lu, X., Wei, B., Liu, K., Wang, K.: Circuit Bootstrapping: Faster and Smaller. In: Joye, M., Leander, G. (eds.) Advances in Cryptology – EUROCRYPT 2024. pp. 342–372. Springer Nature Switzerland, Cham (2024)
53. Xiang, B., Zhang, J., Deng, Y., Dai, Y., Feng, D.: Fast blind rotation for bootstrapping fhes. In: Annual International Cryptology Conference. pp. 3–36. Springer (2023)
54. Xiang, B., Zhang, J., Wang, K., Deng, Y., Feng, D.: Ntru-based bootstrapping for mk-fhes without using overstretched parameters. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 241–270. Springer (2024)
55. Zama: TFHE-rs: A Pure Rust Implementation of the TFHE Scheme for Boolean and Integer Arithmetics Over Encrypted Data (2022), https://github.com/zama-ai/tfhe-rs
56. Zhou, T., Yang, X., Liu, L., Zhang, W., Li, N.: Faster bootstrapping with multiple addends. IEEE Access **6**, 49868–49876 (2018)

A Algorithms

Algorithm 2: SampleExtract: Extract constant term from GLWE to LWE

Input: $\mathrm{CT}_{\mathrm{in}} = (a_0, \ldots, a_{k-1}, b) \in R_q^{k+1}$, where $a_\alpha(X) = \sum_{t=0}^{N-1} a_\alpha[t] X^t$, $b(X) = \sum_{t=0}^{N-1} b[t] X^t$

Output: $\mathrm{ct}_{\mathrm{out}} \in \mathbb{Z}_q^{kN+1}$: an LWE of p_0 under $\mathbf{s} \in \mathbb{Z}_q^{kN}$

for $i \leftarrow 0$ **to** $kN-1$ **do**
- $\alpha \leftarrow \lfloor i/N \rfloor$;
- $j \leftarrow i \bmod N$;
- $t \leftarrow (N-j) \bmod N$;
- $\mathrm{sign} \leftarrow \begin{cases} +1, & j = 0 \\ -1, & j > 0 \end{cases}$
- $a_{\mathrm{out},i} \leftarrow \mathrm{sign} \cdot a_\alpha[t]$

$\mathrm{ct}_{\mathrm{out}} \leftarrow (a_{\mathrm{out},0}, \ldots, a_{\mathrm{out},kN-1}, b[0])$

return $\mathrm{ct}_{\mathrm{out}}$

Algorithm 3: $\mathsf{LWEKeySwitch}_{\mathbf{s}\to\mathbf{s}'}$: Switch the secret key of a LWE cipher from s to s'

Input: $\mathbf{c} = (\mathbf{a}, b) \in \mathbb{Z}_q^{n+1}$: an LWE encryption of m under old key $\mathbf{s} \in \mathbb{Z}_q^n$

Input: $\mathrm{KS}_{\mathbf{s}\to\mathbf{s}'}[i] = \left(\mathrm{LWE}_{\mathbf{s}'}(\frac{q}{B^1} \cdot s_i), \ldots, \mathrm{LWE}_{\mathbf{s}'}(\frac{q}{B^\ell} \cdot s_i)\right)$ for $i = 1, \ldots, n$ with base B and length ℓ, under new key $\mathbf{s}' \in \mathbb{Z}_q^{n'}$

Output: $\mathbf{c}' = (\mathbf{a}', b') \in \mathbb{Z}_q^{n'+1}$: an LWE encryption of m under $\mathbf{s}'$

for $i = 1$ **to** n **do**
- Decompose a_i as $a_i = \sum_{j=1}^{\ell} a'_{i,j} \cdot \frac{q}{B^j} + e'_i$ with $\|a'_{i,j}\|_\infty \le \frac{B}{2}$ and $|e'_i| \le \frac{q}{2B^\ell}$

$\mathbf{c}' \leftarrow \mathrm{LWE}^0_{\mathbf{s}'}(b)$ **for** $i = 1$ **to** n **do**
- $\mathbf{c}' \leftarrow \mathbf{c}' - \sum_{j=1}^{\ell} a'_{i,j} \cdot \mathrm{KS}_{\mathbf{s}\to\mathbf{s}'}[i][j]$

return $\mathbf{c}'$

Algorithm 4: $\mathsf{ModSwitch}_{Q_1 \to Q_2}$: Switch the ciphertext modulus

Input: $\mathbf{c} = (\mathbf{a}, b) \in \mathbb{Z}_{Q_1}^{n+1}$: an LWE encryption of m under key $\mathbf{s} \in \mathbb{Z}_{Q_1}^{n}$
Input: $Q_1, Q_2 \in \mathbb{Z}_{>0}$ with $Q_2 < Q_1$
Output: $\mathbf{c}' = (\mathbf{a}', b') \in \mathbb{Z}_{Q_2}^{n+1}$: an LWE encryption of m under the *same* key $\mathbf{s}$

$\delta \leftarrow \frac{Q_2}{Q_1}$
for $i = 1$ **to** n **do**
  $a_i' \leftarrow \lfloor \delta \cdot a_i \rceil \bmod Q_2$
$b' \leftarrow \lfloor \delta \cdot b \rceil \bmod Q_2$
return $\mathbf{c}' = (\mathbf{a}', b')$

Algorithm 5: $\mathsf{PBSmanyLUT}$

Input: $\mathbf{c}_{\mathsf{in}} = \mathrm{LWE}_{\mathbf{s}}(m \cdot \Delta_{\mathsf{in}}) = (a_1, \ldots, a_n, a_{n+1} = b) \in \mathbb{Z}_q^{n+1}$ under $\mathbf{s} = (s_1, \ldots, s_n)$ where $(\beta, m') \leftarrow \mathsf{PTModSwitch}_q(m, \Delta_{\mathsf{in}}, \vartheta)$
Input: $\mathrm{BSK} = (\mathrm{GGSW}_{\mathbf{S}'}(s_i))_{1 \le i \le n}$ under $\mathbf{S}' = (S_1', \ldots, S_k')$ with decomposition base B_{pbs} and level ℓ_{pbs}
Input: $P_{(f_1, \ldots, f_{2^\vartheta})}$: a redundant LUT for $f_1, \ldots, f_{2^\vartheta}$
Output: $\mathbf{c}_1, \ldots \mathbf{c}_{2^\vartheta}$ where $\mathbf{c}_j = \mathrm{LWE}_{\mathbf{s}'}((-1)^\beta \cdot f_j(m') \cdot \Delta_{\mathsf{out}})$

and $\mathbf{s}'$ is the LWE secret key corresponding to $\mathbf{S}'$ **for** $i = 1$ *to* $n+1$ **do**
  $a_i' \leftarrow \left[\left\lfloor \frac{a_i \cdot 2N \cdot 2^{-\vartheta}}{q} \right\rceil \cdot 2^\vartheta \right]_{2N}$
$\mathbf{C}_{(f_1, \ldots, f_{2^\vartheta})} \leftarrow \mathrm{GLWE}^0_{\mathbf{S}'}(P_{(f_1, \ldots, f_{2^\vartheta})})$
$\mathbf{C} \leftarrow \mathsf{BlindRotate}\left(\mathbf{C}_{(f_1, \ldots, f_{2^\vartheta})}, \{a_i'\}_{i=1}^{n+1}, \mathrm{BSK}\right)$
for $j = 1$ *to* 2^ϑ **do**
  $\mathbf{c}_j \leftarrow \mathsf{SampleExtract}_{j-1}(\mathbf{C})$

Algorithm 6: $\mathsf{GMS}_{q_{\text{in}} \to q_{\text{out}}}$: General Companion Modulus Switching with Mean Compensation

Input: $\mathbf{c} = (\mathbf{a}, b) \in \mathbb{Z}_{q_{\text{in}}}^{n+1}$: an LWE encryption of m under key $\mathbf{s} \in \{0,1\}^n$

Input: $q_{\text{out}} < q_{\text{in}}$, rounding parameter $v \in \mathbb{Z}_{>0}$, budget $d \in \mathbb{Z}_{\geq 0}$

Output: $\mathbf{c}^* = (\mathbf{a}^*, b^*) \in \mathbb{Z}_{q_{\text{out}}}^{n+1}$

$\alpha \leftarrow \frac{q_{\text{in}}}{q_{\text{out}}}$

$A \leftarrow 0$

for $i = 1$ **to** n **do**

  $a'_i \leftarrow \lfloor \alpha^{-1} \cdot a_i \rceil$

  $a''_i \leftarrow \lfloor a'_i \rceil_v$

  if $\widetilde{a}_i \neq a'_i$ ***and*** $d > 0$ **then**

    $\overline{\overline{a}}_i = a_i - \alpha \cdot a''_i$

    $A \leftarrow A + \overline{\overline{a}}_i \cdot \alpha$

    $a^*_i \leftarrow a''_i$

    $d \leftarrow d - 1$

  else

    $\overline{a}_i = a_i - \alpha \cdot a'_i$

    $A \leftarrow A + \overline{a}_i \cdot \alpha$

    $a^*_i \leftarrow a'_i$

$b_{\text{comp}} \leftarrow b - \mu_s \cdot A$

$b^* \leftarrow \lfloor \alpha^{-1} \cdot b_{\text{comp}} \rceil$

return $\mathbf{c}^* = (\mathbf{a}^*, b^*)$

B Proofs

B.1 Proof of Lemma 1

Proof. **Step 1: Defining the $\mathcal{R}_N$-module structure on $\mathcal{R}_{q/2}$.**
Let $\mathcal{R}_N = \mathbb{Z}[Y]/(Y^N + 1)$, where $N = q/(2\tau)$. We define a ring homomorphism $h : \mathcal{R}_N \to \mathcal{R}_{q/2}$ by mapping $Y \mapsto X^\tau$. This is a well-defined homomorphism because $Y^N + 1$ maps to $X^{N\tau} + 1 = X^{q/2} + 1 \equiv 0$ in $\mathcal{R}_{q/2}$. Explicitly, for any $F(Y) \in \mathcal{R}_N$, $h(F(Y)) = F(X^\tau)$ in $\mathcal{R}_{q/2}$. This homomorphism enables us to define the scalar multiplication $\cdot : \mathcal{R}_N \times \mathcal{R}_{q/2} \to \mathcal{R}_{q/2}$. For any $r \in \mathcal{R}_N$ and $m \in \mathcal{R}_{q/2}$, we set:

$$r \cdot a = h(r)a$$

where $h(r)a$ represents the standard ring multiplication of $h(r)$ and a in $\mathcal{R}_{q/2}$, which is defined as the polynomial multiplication in $Z[X]$ followed by a reduction modulo $(X^{q/2} + 1)$.

Now, we need to verify that this operation satisfies the four axioms of an R-module:

- Distributivity over module addition holds from the distributive property of polynomial multiplication: For any $r \in \mathcal{R}_N$ and $a, b \in \mathcal{R}_{q/2}$, $r \cdot (a + b) = h(r)(a + b) = h(r)a + h(r)b = r \cdot a + r \cdot b$.
- Distributivity over ring addition holds from the distributive property of polynomial multiplication and the homomorphism property of h: For any $r, r' \in \mathcal{R}_N$ and $a \in \mathcal{R}_{q/2}$, $(r + r') \cdot a = h(r + r')a = (h(r) + h(r'))a = h(r)a + h(r')a = r \cdot a + r' \cdot a$.
- Associativity of scalar multiplication holds from the associative property of polynomial multiplication and the isomorphism of h: For $r, r' \in \mathcal{R}_N$ and $a \in \mathcal{R}_{q/2}$, $(rr') \cdot a = h(rr')a = (h(r)h(r'))a = h(r)(h(r')a) = r \cdot (r' \cdot a)$.
- Identity element action holds from the isomorphism of h, which maps the identity of $\mathcal{R}_N$ to the identity of $\mathcal{R}_{q/2}$: For any $a \in \mathcal{R}_{q/2}$, $1 \cdot a = h(1)a = 1a = a$.

Therefore, with this defined scalar multiplication, $\mathcal{R}_{q/2}$ is an $\mathcal{R}_N$-module.

Step 2: Proving that $\mathcal{R}_{q/2}$ is a free $\mathcal{R}_N$-module with basis $\{X^i\}_{0 \le i \le \tau - 1}$.
Let $B = \{1, X, X^2, \ldots, X^{\tau-1}\}$. We need to show that B is a basis for $\mathcal{R}_{q/2}$ over $\mathcal{R}_N$. This requires demonstrating both that B spans $\mathcal{R}_{q/2}$ and that B is linearly independent over $\mathcal{R}_N$.

Let $F(X) \in \mathcal{R}_{q/2}$. By definition, $F(X)$ is a polynomial in $\mathbb{Z}[X]$ considered modulo $(X^{q/2} + 1)$. Any term $a_k X^k$ in $F(X)$ can be written as $a_k X^{j\tau+i}$, where $0 \le i < \tau$ and j are some integers. Thus, we can group the terms of $F(X)$ based on their exponent modulo τ (Note that the upper limit of j is $N - 1$ because $X^{N\tau} = X^{q/2} \equiv -1 \pmod{X^{q/2} + 1}$. So powers of X^τ higher than $N - 1$ can be reduced.):

$$F(X) = \sum_{k=0}^{q/2-1} a_k X^k = \sum_{i=0}^{\tau-1} \left(\sum_{j=0}^{N-1} a_{j\tau+i} X^{j\tau} \right) X^i.$$

Let $F_i(Y) = \sum_{j=0}^{N-1} a_{j\tau+i} Y^j \in \mathcal{R}_N$, then by the definition of the homomorphism h, we have that $h(F_i(Y)) = \sum_{j=0}^{N-1} a_{j\tau+i} X^{j\tau}$. Therefore, $F(X)$ can be expressed as a linear combination of elements in B with coefficients in $\mathcal{R}_N$: $F(X) = F_0(Y) \cdot 1 + F_1(Y) \cdot X + \cdots + F_{\tau-1}(Y) \cdot X^{\tau-1}$. This confirms that B spans $\mathcal{R}_{q/2}$ as an $\mathcal{R}_N$-module.

Then suppose we have a linear combination of elements in B that equals zero in $\mathcal{R}_{q/2}$: $F_0(Y) \cdot 1 + F_1(Y) \cdot X + \cdots + F_{\tau-1}(Y) \cdot X^{\tau-1} = 0$, where each $F_i(Y) \in \mathcal{R}_N$. This means that the polynomial $F(X) = \sum_{i=0}^{\tau-1} h(F_i(Y)) X^i$ is divisible by $X^{q/2} + 1$ in $\mathbb{Z}[X]$. Suppose $F_i(Y) = \sum_{j=0}^{N-1} a_{ij} Y^j$ for some integers a_{ij}, then that $h(F_i(Y))$ is a polynomial in X^τ of degree at most $(N-1)\tau$ that equals to $\sum_{j=0}^{N-1} a_{ij} X^{j\tau}$. Then

$$F(X) = \sum_{i=0}^{\tau-1} \left(\sum_{j=0}^{N-1} a_{ij} X^{j\tau} \right) X^i = \sum_{i=0}^{\tau-1} \sum_{j=0}^{N-1} a_{ij} X^{j\tau+i}.$$

The exponents $j\tau + i$ in this sum are all distinct for $0 \le i < \tau$ and $0 \le j < N$, and the highest possible exponent is $(N-1)\tau + (\tau-1) = N\tau - \tau + \tau - 1 = N\tau - 1 = q/2 - 1$. Therefore, the only polynomial of this form that is divisible by $X^{q/2} + 1$ is the zero polynomial. This implies that all coefficients a_{ij} in $P(X)$ must be zero. Since $F_i(Y) = \sum_{j=0}^{N-1} a_{ij} Y^j$, it follows that $F_i(Y) = 0$ in $\mathcal{R}_N$ for all i. This establishes that B is linearly independent over $\mathcal{R}_N$.

Since B both spans $\mathcal{R}_{q/2}$ and is linearly independent over $\mathcal{R}_N$, it is a basis for $\mathcal{R}_{q/2}$ as an $\mathcal{R}_N$-module. Therefore, $\mathcal{R}_{q/2}$ is a free $\mathcal{R}_N$-module. □

B.2 Proof of Theorem 2

We analyze correctness and noise growth step by step, following Algorithm 1.

Proof. **Accumulator Initialization (line 1).** The test vector is encoded as a trivial vectorized ciphertext $\mathrm{GLWE}_{\mathsf{vec}}(X^{-b} \cdot tv)$, which introduces no additional noise. By construction, tv encodes a negacyclic function f so that the desired value $f(m)$ will be aligned to the constant coefficient after blind rotation.

Blind Rotation via Vectorized CMux (lines 2–5). Each iteration updates the accumulator according to the input coefficient a_i and the encrypted selector $\mathrm{GGSW}_{\mathsf{vec}}(s_i)$. The candidate accumulators $ACC^{(0)} = ACC$ and $ACC^{(1)} = X^{a_i} \cdot ACC$ are related by a monomial multiplication in the base ring, which does not introduce noise. The update is then performed by a vectorized CMux:

$$\mathrm{CMux}_{\mathsf{vec}}(\overline{\mathbf{C}}, \overline{\mathbf{C}}^{(0)}, \overline{\mathbf{C}}^{(1)}) = \overline{\mathbf{C}} \boxdot_{\mathsf{vec}} \left(\overline{\mathbf{C}}^{(1)} - \overline{\mathbf{C}}^{(0)} \right) + \overline{\mathbf{C}}^{(0)}.$$

Under our optimization (Section 4, constants only in the first vector component), the vectorized CMux reduces to τ independent base CMux gates, one per component. After n iterations, the accumulator equals $\mathrm{GLWE}_{\mathsf{vec}}\left(X^{-b+\sum_{i=1}^{n} a_i s_i} \cdot tv\right) = \mathrm{GLWE}_{\mathsf{vec}}\left(X^{-(\Delta m+e)} \cdot tv\right)$, which aligns $f(m)$ to the constant term in the first component.

The cumulative variance contributed by the n CMux operations is (by Theorem 4 in [20]):

$$\sigma_{\mathsf{br}}^2 = n\ell_{\mathsf{br}}(k+1)N\frac{B_{\mathsf{br}}^2+2}{12}\sigma_{\mathsf{GLWE}}^2 + n\frac{q^2 - B_{\mathsf{br}}^{2\ell_{\mathsf{br}}}}{24B_{\mathsf{br}}^{2\ell_{\mathsf{br}}}}\left(1+\frac{kN}{2}\right) + \frac{nkN}{32} + \frac{n}{16}\left(1-\frac{kN}{2}\right)^2,$$

where ℓ_{br} and B_{br} are the gadget parameters used in blind rotation and σ_{GLWE}^2 is the variance of the base GLWE encryption.

Sample Extraction (line 6). We extract the constant term of the first component ACC_0, obtaining an LWE ciphertext of $f(m)$ under the re-ordered secret key $\mathbf{s}'$ induced by $\mathbf{S}_{kN}$. This step is algebraic (a coefficient re-indexing) and does not add noise.

Key Switching (line 7). We convert $\mathrm{LWE}_{kN,Q}(f(m))$ to $\mathrm{LWE}_{n,Q}(f(m))$ via a standard LWE-to-LWE key-switch. The variance added by key switching is

$$\sigma_{\mathsf{ks}}^2 = \ell_{\mathsf{ks}}\, kN\left(\frac{B_{\mathsf{ks}}^2+2}{12}\right)\sigma_{\mathsf{LWE}}^2 \;+\; kN\left(\frac{q^2 - B_{\mathsf{ks}}^{2\ell_{\mathsf{ks}}}}{24B_{\mathsf{ks}}^{2\ell_{\mathsf{ks}}}} + \frac{1}{12}\right),$$

where ℓ_{ks} and B_{ks} are the gadget parameters used in key switching and σ_{LWE}^2 is the base LWE noise variance.

Modulus Switching (line 8). Finally, modulus switching rounds $\mathrm{LWE}_{n,Q}(f(m))$ to $\mathrm{LWE}_{n,q}(f(m))$, contributing the rounding variance

$$\sigma_{\mathsf{ms}}^2 \;=\; \frac{kN+4}{48} - \frac{(n+1)q^2}{12Q^2} \quad [20,45].$$

Total Output Noise. Combining the above, the total output variance of programmable bootstrapping is

$$\sigma_{\mathsf{pbs}}^2 \;=\; \frac{q^2}{Q^2}(\sigma_{\mathsf{br}}^2 + \sigma_{\mathsf{ks}}^2) \;+\; \sigma_{\mathsf{ms}}^2.$$

□

B.3 Proof of Lemma 3

Proof. We adopt the primal attack as the reference point for security. Following [2], for an LWE instance with $m = O(n)$ samples and noise standard deviation σ, the primal success condition at BKZ blocksize β is

$$\sigma\sqrt{\beta} \;\le\; \delta_\beta^{2\beta-2-(m+n)} \cdot Q^{\frac{m}{m+n+1}} \cdot (2\sigma)^{\frac{n}{m+n+1}}, \tag{12}$$

where $\delta_\beta = \left((\pi\beta)^{1/\beta}\frac{\beta}{2\pi e}\right)^{1/(2(\beta-1))}$, and cost dominated by β: $O(2^{0.292\beta})$ classically / $O(2^{0.265\beta})$ quantumly. For a fixed security level, treating β as constant, the security condition simplifies to

$$kN \underset{\mathrm{GLWE}}{\ge} O(n) \underset{\mathrm{LWE}}{\ge} O\left(\log\left(\frac{Q}{\sigma}\right)\right). \tag{13}$$

The first inequality enforces the security constraint for GLWE in $\mathcal{R}^k_{Q,N}$, while the second enforces the constraint for LWE in $\mathbb{Z}^n_Q$. Thus completing the proof of Lemma 3. □

B.4 Proof of Lemma 4

Proof. By Theorem 2, the (per-coefficient) noise variance in a refreshed $\mathcal{R}_{Q,N}$ sub-ciphertext can be upper bounded by

$$\sigma^2_{\mathsf{coeff}} \;=\; O\Big(\Big(\tfrac{q^2}{Q^2}\, B^2 \ell\, n\sigma^2 + 1\Big)\, kN\Big). \tag{14}$$

Let E denote a single coefficient error, and assume E is (upper bounded by) a centered Gaussian with variance $\sigma^2_{\mathsf{coeff}}$ from Eq. (14). Let $\Delta = \Theta(q/t)$ be the decryption threshold per coefficient. Then

$$\Pr[|E| > \Delta] \;=\; \mathrm{erfc}\Big(\tfrac{\Delta}{\sqrt{2}\sigma_{\mathsf{coeff}}}\Big).$$

To avoid asymptotic approximations, we use a standard Mills-type bound: for all $x > 0$,

$$\mathrm{erfc}(x) \;\le\; \frac{e^{-x^2}}{\sqrt{\pi}\, x}. \tag{15}$$

Setting $x := \frac{\Delta}{\sqrt{2}\sigma_{\mathsf{coeff}}}$ yields

$$\Pr[|E| > \Delta] \;\le\; \frac{\sqrt{2}\sigma_{\mathsf{coeff}}}{\sqrt{\pi}\,\Delta} \cdot \exp\Big(-\tfrac{\Delta^2}{2\sigma^2_{\mathsf{coeff}}}\Big). \tag{16}$$

Applying a union bound over the N coefficients of a single refreshed sub-ciphertext gives

$$\mathrm{DFR} \;\le\; N \cdot \Pr[|E| > \Delta] \;\le\; N \cdot \frac{\sqrt{2}\sigma_{\mathsf{coeff}}}{\sqrt{\pi}\,\Delta} \cdot \exp\Big(-\tfrac{\Delta^2}{2\sigma^2_{\mathsf{coeff}}}\Big).$$

Enforcing $\mathrm{DFR} \le 2^{-r}$ is therefore implied by

$$r \ln 2 \;\le\; \frac{\Delta^2}{2\sigma^2_{\mathsf{coeff}}} \;-\; \ln N \;-\; \ln\Big(\tfrac{\sqrt{2}\sigma_{\mathsf{coeff}}}{\sqrt{\pi}\,\Delta}\Big). \tag{17}$$

We now upper bound the logarithmic correction term. Since $\Delta = \Theta(q/t)$ and $\sigma^2_{\mathsf{coeff}} = O\Big(\big(\tfrac{q^2}{Q^2} B^2 \ell n \sigma^2 + 1\big) kN\Big)$, we have

$$\ln\Big(\tfrac{\sqrt{2}\sigma_{\mathsf{coeff}}}{\sqrt{\pi}\,\Delta}\Big) = O\Big(\ln\big(\tfrac{\sigma_{\mathsf{coeff}}\, t}{q}\big)\Big),$$

which is lower-order compared to the quadratic term $\Delta^2/\sigma^2_{\mathsf{coeff}}$ in the parameter regimes of interest. Thus, ignoring this additive lower-order term (or equivalently absorbing it into the constant factors hidden in $O(\cdot)$), Eq. (17) yields the simplified sufficient condition

$$r \ln 2 + \ln N \;\le\; O\Big(\tfrac{\Delta^2}{\sigma^2_{\mathsf{coeff}}}\Big). \tag{18}$$

Substituting $\Delta = \Theta(q/t)$ and Eq. (14) into Eq. (18) gives

$$r \ln 2 + \ln N \;\le\; O\left(\frac{q^2/t^2}{\left(\frac{q^2}{Q^2} B^2 \ell n \sigma^2 + 1\right) kN} \right).$$

Rearranging yields

$$Q = O\left(\sigma \, t \, q \, B \cdot \sqrt{\frac{\ell n k N \, (r \ln 2 + \ln N)}{q^2 - t^2 k N \, (r \ln 2 + \ln N)}} \right), \tag{19}$$

which is Eq. (5) with the dependence on r made explicit (replacing $\ln N$ by $r \ln 2 + \ln N$ up to constant factors).

Finally, the denominator in Eq. (19) must be positive, giving the necessary constraint in Eq. (5). □

B.5 Proof of Theorem 3

Proof. Eq. (5) implies $t \le \Theta\left(\frac{q}{\sqrt{kN \ln N}}\right)$. Choosing a constant slack $\alpha \in (0, 1)$ yields the parameterization in Eq. (6).

TFHE. Set $k = 1$ and $N = q/2$, so Eq. (6) becomes

$$t = \alpha \cdot \frac{q}{\sqrt{(q/2)\ln(q/2)}} = \alpha \sqrt{\frac{2q}{\ln(q/2)}}.$$

Rearranging gives

$$q = \frac{t^2}{2\alpha^2} \ln(q/2) = \Theta(t^2 \log q).$$

This implicit equation solves to a Lambert-W expression on the -1 branch,

$$q = \Theta\left(-t^2 \, W_{-1}\left(-\Theta\left(\tfrac{1}{t^2}\right)\right)\right),$$

and using the standard asymptotic $-W_{-1}(-\varepsilon) = \Theta(\ln(1/\varepsilon))$ as $\varepsilon \to 0^+$ yields $q = \Theta(t^2 \log t)$ as $t \to \infty$.

Ours. Set $k = 1$ and $N = \Theta(n)$, so $\log N = \Theta(\log n)$ and $kN = \Theta(n)$. Then Eq. (6) gives

$$t = \alpha \cdot \frac{q}{\sqrt{\Theta(n \log n)}} \quad \Longleftrightarrow \quad q = \Theta\left(t \sqrt{n \log n}\right),$$

which is Eq. (8).

B.6 Proof of Corollary 1

Proof. The external modulus scaling follows by substituting $B = \Theta(Q^{1/\ell})$ into Eq. (5) and rearranging to get Eq. (9), then plugging in the $q(t)$ relations from Theorem 3.

The runtime and BSK-size scalings follow by substituting the same $q(t)$ relations and the relevant choices of (k, τ, N) into Eq. (10) and $S_{\mathsf{BSK}} = O(nN \log(q\sqrt{n}))$, respectively, and simplifying polylog factors. □

B.7 Proof of Lemma 6

Proof. We employ an exchange argument. Suppose the sequence of prime factors $\mathbf{p} = (p_1, \ldots, p_k)$ is not sorted in non-decreasing order. Then, there exists at least one adjacent pair at index j such that $p_j > p_{j+1}$.

Consider a modified sequence $\mathbf{p}'$ where these two factors are swapped, the change in the total cost N_{CMux} depends solely on the terms at indices j and $j+1$. We calculate the difference Δ_{cost} between the cost contributions of the swapped pair and the original pair:

$$\Delta_{cost} = \frac{1}{\pi_{j-1}^2} \left(\left[\left(1 - \frac{1}{p_{j+1}}\right) + \frac{1}{p_{j+1}^2} \left(1 - \frac{1}{p_j}\right) \right] - \left[\left(1 - \frac{1}{p_j}\right) + \frac{1}{p_j^2} \left(1 - \frac{1}{p_{j+1}}\right) \right] \right).$$

We factor out the term $(\frac{1}{p_{j+1}} - \frac{1}{p_j})$ from the expression.

$$\Delta_{cost} = -\frac{1}{\pi_{j-1}^2} \left(\frac{1}{p_{j+1}} - \frac{1}{p_j} \right) \left(1 - \frac{1}{p_j}\right) \left(1 - \frac{1}{p_{j+1}}\right).$$

Since $p_j > p_{j+1}$, it follows that $\frac{1}{p_{j+1}} > \frac{1}{p_j}$, so the term $(\frac{1}{p_{j+1}} - \frac{1}{p_j})$ is positive. Additionally, for any prime factor p_i, the term $(1 - 1/p)$ is strictly positive. Therefore, the entire expression is strictly negative ($\Delta_{cost} < 0$).

This confirms that swapping any adjacent pair where $p_j > p_{j+1}$ strictly reduces the total cost. Consequently, the minimum complexity is achieved when the factors are sorted in non-decreasing order. □

B.8 Proof of Lemma 7

Proof. Since the order of the coefficients does not affect the noise distribution, we assume without loss of generality that the first d coefficients are influenced by GMS. Let $\mathbf{c} = (a_1, a_2, \ldots, a_n, b)$ be the input ciphertext and $\mathbf{c}^* = (a_1'', \ldots, a_d'', a_{d+1}', \ldots, a_n', b^*)$ be the output of GMS, where

$$a_i'' = \lfloor \lfloor \alpha^{-1} \cdot a_i \rceil \rceil_v = \alpha^{-1} \cdot a_i + \overline{\overline{a}}_i,$$

$$a_i' = \lfloor \alpha^{-1} \cdot a_i \rceil = \alpha^{-1} \cdot a_i + \overline{a}_i,$$

and

$$b^* = \lfloor \alpha^{-1} \cdot \Big(b - \mu_s \cdot \big(\sum_{i=1}^{d} \overline{\overline{a}}_i \cdot \alpha + \sum_{i=d+1}^{n} \overline{a}_i \cdot \alpha \big) \Big) \rceil$$
$$= \alpha^{-1} \cdot \Big(b - \mu_s \cdot \big(\sum_{i=1}^{d} \overline{\overline{a}}_i \cdot \alpha + \sum_{i=d+1}^{n} \overline{a}_i \cdot \alpha \big) \Big) + \overline{b}.$$

We decrypt $\mathbf{c}^*$ to identify the corresponding noise contribution:

$$\begin{aligned}
&\big\langle (a''_1, \ldots, a''_d, a'_{d+1}, \ldots, a'_n, b^*), \mathbf{s} \big\rangle \\
&= b^* + \sum_{i=1}^{d} a''_i s_i + \sum_{i=d+1}^{n} a'_i s_i \\
&= \alpha^{-1} \cdot b - \alpha^{-1} \cdot \mu_s \cdot \Big(\sum_{i=1}^{d} \overline{\overline{a}}_i \cdot \alpha + \sum_{i=d+1}^{n} \overline{a}_i \cdot \alpha \Big) + \overline{b} \\
&\quad + \sum_{i=1}^{d} (\alpha^{-1} \cdot a_i + \overline{\overline{a}}_i) s_i + \sum_{i=d+1}^{n} (\alpha^{-1} \cdot a_i + \overline{a}_i) s_i \\
&= \alpha^{-1} \Big(b - \sum_{i=1}^{n} a_i s_i \Big) + \overline{b} + \sum_{i=1}^{d} \overline{\overline{a}}_i (s_i - \mu_s) + \sum_{i=d+1}^{n} \overline{a}_i (s_i - \mu_s) \\
&= \alpha^{-1} \Delta m + \alpha^{-1} e_{in} + \overline{b} + \sum_{i=1}^{d} \overline{\overline{a}}_i (s_i - \mu_s) + \sum_{i=d+1}^{n} \overline{a}_i (s_i - \mu_s).
\end{aligned} \tag{20}$$

Consequently, the variance introduced by modulus switching is

$$\begin{aligned}
Var_{GMS} &= Var\Big(\alpha^{-1} e_{in} + \overline{b} + \sum_{i=1}^{d} \overline{\overline{a}}_i (s_i - \mu_s) + \sum_{i=d+1}^{n} \overline{a}_i (s_i - \mu_s) \Big) \\
&= Var(\alpha^{-1} e_{in}) + Var(\overline{b}) \\
&\quad + d \cdot \Big(Var(\overline{\overline{a}}_i) Var(s_i - \mu_s) + Var(\overline{\overline{a}}_i) \mathbb{E}^2(s_i - \mu_s) + \mathbb{E}^2(\overline{\overline{a}}_i) Var(s_i - \mu_s) \Big) \\
&\quad + (n - d) \cdot \Big(Var(\overline{a}_i) Var(s_i - \mu_s) + Var(\overline{a}_i) \mathbb{E}^2(s_i - \mu_s) + \mathbb{E}^2(\overline{a}_i) Var(s_i - \mu_s) \Big) \\
&= \frac{\sigma_{in}^2}{\alpha^2} + Var(\overline{b}) + d \cdot \big(Var(\overline{\overline{a}}_i) + \mathbb{E}^2(\overline{\overline{a}}_i) \big) Var(s_i) \\
&\quad + (n - d) \cdot \big(Var(\overline{a}_i) + \mathbb{E}^2(\overline{a}_i) \big) Var(s_i).
\end{aligned} \tag{21}$$

Since $\overline{a}_i, \overline{b} \in \frac{q_{out}}{q_{in}} \mathcal{U}\Big(\big[-\frac{q_{in}}{2q_{out}}, \frac{q_{in}}{2q_{out}} \big) \Big)$, we have

$$\alpha \mathbb{E}(\overline{a}_i) = \frac{1}{\alpha} \sum_{i=-\alpha/2}^{\alpha/2-1} i = -\frac{1}{2},$$

$$\mathbb{E}(\overline{a}_i) = \mathbb{E}(\overline{b}) = -\frac{1}{2\alpha},$$

$$\alpha^2 Var(\overline{a}_i) = \frac{1}{\alpha}\sum_{i=-\alpha/2}^{\alpha/2-1}\left(i+\frac{1}{2}\right)^2 = \frac{\alpha^2-1}{12},$$

$$Var(\overline{a}_i) = Var(\overline{b}) = \frac{1}{12} - \frac{1}{12\alpha^2}.$$

For $\overline{\overline{a}}_i \in \frac{q_{out}}{q_{in}}\mathcal{U}\Big(\Big(-\frac{vq_{in}}{2q_{out}}, -\frac{q_{in}}{2q_{out}}\Big) \cup \Big[\frac{q_{in}}{2q_{out}}, \frac{vq_{in}}{2q_{out}}\Big)\Big)$, we obtain

$$\alpha\mathbb{E}(\overline{\overline{a}}_i) = \frac{1}{(v-1)\alpha-1}\left(\sum_{i=-\frac{v\alpha}{2}+1}^{-\frac{\alpha}{2}-1} i + \sum_{i=\frac{\alpha}{2}}^{\frac{v\alpha}{2}-1} i\right) = \frac{\alpha}{2((v-1)\alpha-1)},$$

$$\mathbb{E}(\overline{\overline{a}}_i) = \frac{1}{2((v-1)\alpha-1)},$$

$$\begin{aligned}\alpha^2 Var(\overline{\overline{a}}_i) = \frac{1}{(v-1)\alpha-1}\Bigg(&\sum_{i=-\frac{v\alpha}{2}+1}^{-\frac{\alpha}{2}-1}\left(i-\frac{\alpha}{2((v-1)\alpha-1)}\right)^2 \\ &+ \sum_{i=\frac{\alpha}{2}}^{\frac{v\alpha}{2}-1}\left(i-\frac{\alpha}{2((v-1)\alpha-1)}\right)^2\Bigg) \\ = \frac{(v^3-1)\alpha^3 - 3v^2\alpha^2 + 2(v-1)\alpha}{12(v-1)\alpha-12} &- \frac{\alpha^2}{4((v-1)\alpha-1)^2},\end{aligned}$$

$$Var(\overline{\overline{a}}_i) = \frac{(v^3-1)\alpha^3 - 3v^2\alpha^2 + 2(v-1)\alpha}{12(v-1)\alpha^3-12\alpha^2} - \frac{1}{4((v-1)\alpha-1)^2}.$$

Finally, since each s_i follows a Bernoulli distribution over $\{0,1\}$,

$$Var(s_i) = \frac{1}{4}.$$

Substituting all the above expressions, we obtain

$$Var_{GMS} = \frac{\sigma_{in}^2}{\alpha^2} + \sigma_{gms}^2$$

Where σ_{gms}^2 is exoressed as:

$$\begin{aligned}\sigma_{gms}^2 = &\left(\frac{1}{12} - \frac{1}{12\alpha^2}\right) \\ &+ \frac{d}{4}\cdot\left(\frac{(v^3-1)\alpha^3 - 3v^2\alpha^2 + 2(v-1)\alpha}{12\alpha^2((v-1)\alpha-1)}\right) \\ &+ \frac{n-d}{4}\cdot\left(\frac{\alpha^2+2}{12\alpha^2}\right)\end{aligned} \tag{22}$$

□

B.9 Proof of Theorem 4

Proof. From the construction of the isomorphism, $F(X) = \Sigma_{0\le i\le \tau-1} F_i(X^\tau)X^i$, and for each $F_i(X^\tau)\cdot X^i$, it follows that:

$$\sigma_a\big(F_i(X^\tau)X^i\big) = F_i((X^\tau)^a)X^{ai}.$$

Divide ai by τ to obtain the quotient and remainder decomposition: $ai = r_i + \tau s_i \quad (0 \le r_i < \tau)$, hence $X^{ai} = X^{r_i}\,(X^\tau)^{s_i}$.

Therefore,

$$\sigma_a\left(\sum_{i=0}^{\tau-1} F_i(X^\tau)X^i\right) = \sum_{i=0}^{\tau-1}(X^\tau)^{s_i}\,F_i((X^\tau)^a)X^{r_i}.$$

Since $\gcd(a,\tau) = 1$ (which follows from $\gcd(a,q) = 1$ and $\tau \mid \frac{q}{2}$), the mapping $i \mapsto r_i$ is a permutation on the set $\{0,\dots,\tau-1\}$, thus yielding that $\sigma_a(F(X))$ has representation that

$$\left(Y^{\lfloor \frac{ak_0}{\tau}\rfloor}F_{k_0}(Y^a),\cdots,Y^{\lfloor \frac{ak_{\tau-1}}{\tau}\rfloor}F_{k_{\tau-1}}(Y^a)\right),$$

which equals to the explicit component-wise expression shown in the lemma. □

C Parameters

Table 7: Parameters table for MBS

Target DFR	$\log m$	n	σ_{LWE}	k	N	σ_{GLWE}	τ	d	ℓ_{ks}	$\log B_{\text{ks}}$	ℓ_{pbs}	$\log B_{\text{pbs}}$	True DFR
2^{-40}	1	486	1.10243e-3	5	256	9.62973e-10	1	0	1	9	1	14	$2^{-40.9667}$
	2	545	3.88236e-4	5	256	4.43607e-10	1	0	1	9	1	15	$2^{-40.8330}$
	3	587	1.82753e-4	3	512	1.29073e-11	1	0	2	9	1	17	$2^{-40.0161}$
	4	637	7.40005e-5	2	1024	2.94036e-16	1	0	2	9	1	23	$2^{-41.0019}$
	5	674	3.78038e-5	1	2048	2.94036e-16	1	0	2	9	1	23	$2^{-40.1024}$
	6	781	8.86848e-6	1	2048	2.94036e-16	1	0	2	9	1	23	$2^{-40.6310}$
	7	788	3.86795e-6	1	2048	2.94036e-16	2	0	2	9	1	23	$2^{-40.7388}$
	8	788	3.86795e-6	1	2048	2.94036e-16	6	0	2	9	1	23	$2^{-41.6206}$
	9	840	1.48614e-6	1	2048	2.94036e-16	11	0	2	9	1	23	$2^{-42.1626}$
	10	918	5.96000e-7	1	2048	2.94036e-16	21	0	2	9	2	15	$2^{-41.0004}$
	11	1024	7.50000e-8	1	2048	2.94036e-16	36	0	3	9	2	15	$2^{-41.0216}$
	12	1024	7.50000e-8	1	2048	2.94036e-16	74	0	3	9	2	15	$2^{-40.0016}$
	13	1024	7.50000e-8	1	2048	2.94036e-16	172	0	3	9	2	15	$2^{-40.3057}$
	14	1088	4.44094e-8	1	2048	2.94036e-16	387	0	3	9	2	15	$2^{-40.0810}$
	15	1163	1.21757e-8	1	2048	2.94036e-16	677	0	3	9	2	15	$2^{-40.0438}$
	16	1280	4.43607e-10	1	2048	2.94036e-16	1417	0	3	9	2	15	$2^{-40.0347}$
	17	1280	4.43607e-10	1	2048	2.94036e-16	3645	0	3	9	2	15	$2^{-40.0095}$
	18	1280	4.43607e-10	1	2048	2.94036e-16	5634	0	3	9	3	11	$2^{-40.0031}$
	19	1280	4.43607e-10	1	2048	2.94036e-16	12746	0	3	9	3	11	$2^{-40.0034}$
	20	1280	4.43607e-10	1	2048	2.94036e-16	31952	0	4	9	3	11	$2^{-40.0010}$
	21	1400	8.97680e-11	1	2048	2.94036e-16	48948	0	4	9	4	9	$2^{-40.0014}$
						...							
	32*	1797	9.92936e-14	1	2048	2.94036e-16	169869312	0	4	10	16	3	$2^{-40.1227}$
2^{-64}	6	720	1.35627e-5	1	2048	2.94036e-16	2	0	2	9	1	23	$2^{-64.0898}$
	7	788	3.86795e-6	1	2048	2.94036e-16	3	0	2	9	1	23	$2^{-75.8821}$
	8	818	2.22155e-6	1	2048	2.94036e-16	6	0	2	9	1	23	$2^{-65.4097}$
	9	918	5.96000e-7	1	2048	2.94036e-16	13	0	2	9	1	23	$2^{-69.2582}$
	10	971	1.96000e-7	1	2048	2.94036e-16	22	0	3	9	2	15	$2^{-64.6343}$
	11	971	1.96000e-7	1	2048	2.94036e-16	48	0	3	9	2	15	$2^{-64.1954}$
	12	1024	7.50000e-8	1	2048	2.94036e-16	95	0	3	9	2	15	$2^{-64.7345}$
	13	1088	4.44094e-8	1	2048	2.94036e-16	201	0	3	9	2	15	$2^{-64.3045}$
	14	1119	2.60130e-8	1	2048	2.94036e-16	425	0	3	9	2	15	$2^{-64.2254}$
	15	1163	1.21757e-8	1	2048	2.94036e-16	871	0	3	9	2	15	$2^{-64.1028}$
	16	1280	4.43607e-10	1	2048	2.94036e-16	1798	0	3	9	2	15	$2^{-64.0296}$
	17	1280	4.43607e-10	1	2048	2.94036e-16	3346	0	3	9	3	11	$2^{-64.0181}$
	18	1280	4.43607e-10	1	2048	2.94036e-16	6988	0	3	9	3	11	$2^{-64.0057}$
	19	1280	4.43607e-10	1	2048	2.94036e-16	16909	0	3	9	3	11	$2^{-64.0013}$
	20	1400	8.97680e-11	1	2048	2.94036e-16	41013	0	4	9	3	11	$2^{-64.0015}$
2^{-80}	6	768	8.76387e-6	1	2048	2.94036e-16	2	0	2	9	1	23	$2^{-103.3133}$
	7	818	2.22155e-6	1	2048	2.94036e-16	3	0	2	9	1	23	$2^{-88.3414}$
	8	818	2.22155e-6	1	2048	2.94036e-16	8	0	2	9	1	23	$2^{-91.5947}$
	9	879	1.96189e-7	1	2048	2.94036e-16	14	0	2	9	1	23	$2^{-86.1522}$
	10	971	1.96000e-7	1	2048	2.94036e-16	30	0	2	9	2	15	$2^{-81.4922}$
	11	971	1.96000e-7	1	2048	2.94036e-16	56	0	3	9	2	15	$2^{-82.2255}$
	12	1024	7.50000e-8	1	2048	2.94036e-16	107	0	3	9	2	15	$2^{-80.4418}$
	13	1088	4.44094e-8	1	2048	2.94036e-16	228	0	3	9	2	15	$2^{-80.3182}$
	14	1119	2.60130e-8	1	2048	2.94036e-16	486	0	3	9	2	15	$2^{-80.1697}$
	15	1163	1.21757e-8	1	2048	2.94036e-16	994	0	3	9	2	15	$2^{-80.0450}$
	16	1280	4.43607e-10	1	2048	2.94036e-16	2036	0	3	9	2	15	$2^{-80.0547}$
	17	1280	4.43607e-10	1	2048	2.94036e-16	3708	0	3	9	3	11	$2^{-80.0012}$
	18	1280	4.43607e-10	1	2048	2.94036e-16	7801	0	3	9	3	11	$2^{-80.0032}$
	19	1280	4.43607e-10	1	2048	2.94036e-16	16797	0	4	9	3	11	$2^{-80.0050}$
	20	1280	4.43607e-10	1	2048	2.94036e-16	37133	0	4	9	4	9	$2^{-80.0031}$
2^{-128}	6	788	3.86795e-6	1	2048	2.94036e-16	2	0	2	9	1	23	$2^{-157.2733}$
	7	818	2.22155e-6	1	2048	2.94036e-16	4	0	2	9	1	23	$2^{-142.9590}$
	8	840	1.48614e-6	1	2048	2.94036e-16	9	0	2	9	1	23	$2^{-134.9866}$
	9	879	1.96189e-7	1	2048	2.94036e-16	20	0	2	9	1	23	$2^{-133.8758}$
	10	971	1.96000e-7	1	2048	2.94036e-16	32	0	3	9	2	15	$2^{-130.7216}$
	11	971	1.96000e-7	1	2048	2.94036e-16	77	0	3	9	2	15	$2^{-128.7437}$
	12	1024	7.50000e-8	1	2048	2.94036e-16	141	0	3	9	2	15	$2^{-128.7315}$
	13	1088	4.44094e-8	1	2048	2.94036e-16	306	0	3	9	2	15	$2^{-128.5797}$
	14	1163	1.21757e-8	1	2048	2.94036e-16	564	0	3	9	2	15	$2^{-128.3322}$
	15	1163	1.21757e-8	1	2048	2.94036e-16	1373	0	3	9	2	15	$2^{-128.0299}$
	16	1221	2.00000e-9	1	2048	2.94036e-16	2702	0	3	9	2	15	$2^{-128.0596}$
	17	1221	2.00000e-9	1	2048	2.94036e-16	4864	0	3	9	3	11	$2^{-128.0086}$
	18	1280	4.43607e-10	1	2048	2.94036e-16	10019	0	3	9	3	11	$2^{-128.0109}$
	19	1280	4.43607e-10	1	2048	2.94036e-16	22600	0	4	9	3	11	$2^{-128.0054}$
	20	1400	8.97680e-11	1	2048	2.94036e-16	39456	0	4	9	4	9	$2^{-128.0301}$

Table 8: Parameters table for SMBS

Target DFR	$\log m$	n	σ_{LWE}	k	N	σ_{GLWE}	τ	d	ℓ_{ks}	$\log B_{\text{ks}}$	ℓ_{pbs}	$\log B_{\text{pbs}}$	True DFR
2^{-40}	6	781	8.86848e-6	1	2048	2.94036e-16	1	0	2	8	1	23	$2^{-40.2611}$
	7	788	3.86795e-6	1	2048	2.94036e-16	2	2	2	9	1	23	$2^{-40.2341}$
	8	788	3.86795e-6	1	2048	2.94036e-16	6	13	2	9	1	23	$2^{-40.0174}$
	9	840	1.48614e-6	1	2048	2.94036e-16	12	40	2	9	1	23	$2^{-40.2930}$
	10	918	5.96000e-7	1	2048	2.94036e-16	24	50	2	9	2	15	$2^{-40.4103}$
	11	1024	7.50000e-8	1	2048	2.94036e-16	36	3	3	9	2	15	$2^{-40.2840}$
	12	1024	7.50000e-8	1	2048	2.94036e-16	76	9	3	9	2	15	$2^{-40.0501}$
	13	1024	7.50000e-8	1	2048	2.94036e-16	176	9	3	9	2	15	$2^{-40.1049}$
	14	1088	4.44094e-8	1	2048	2.94036e-16	400	12	3	9	2	15	$2^{-40.0960}$
	15	1163	1.21757e-8	1	2048	2.94036e-16	680	1	3	9	2	15	$2^{-40.1885}$
	16	1280	4.43607e-10	1	2048	2.94036e-16	1424	2	3	9	2	15	$2^{-40.0512}$
	17	1280	4.43607e-10	1	2048	2.94036e-16	3648	0	3	9	2	15	$2^{-40.0531}$
	18	1280	4.43607e-10	1	2048	2.94036e-16	5640	0	3	9	3	11	$2^{-40.1012}$
	19	1280	4.43607e-10	1	2048	2.94036e-16	12800	0	3	9	3	11	$2^{-40.3216}$
	20	1280	4.43607e-10	1	2048	2.94036e-16	32000	0	4	9	3	11	$2^{-40.0758}$
2^{-64}	6	720	1.35627e-5	1	2048	2.94036e-16	2	0	2	9	1	23	$2^{-64.0898}$
	7	788	3.86795e-6	1	2048	2.94036e-16	4	160	2	9	1	23	$2^{-64.1488}$
	8	818	2.22155e-6	1	2048	2.94036e-16	6	4	2	9	1	23	$2^{-64.1717}$
	9	918	5.96000e-7	1	2048	2.94036e-16	16	105	2	9	1	23	$2^{-64.0138}$
	10	1024	7.50000e-8	1	2048	2.94036e-16	24	31	3	9	2	15	$2^{-64.2860}$
	11	971	1.96000e-7	1	2048	2.94036e-16	48	0	3	9	2	15	$2^{-64.1954}$
	12	1024	7.50000e-8	1	2048	2.94036e-16	96	5	3	9	2	15	$2^{-64.2443}$
	13	1088	4.44094e-8	1	2048	2.94036e-16	204	6	3	9	2	15	$2^{-64.1180}$
	14	1119	2.60130e-8	1	2048	2.94036e-16	432	6	3	9	2	15	$2^{-64.2643}$
	15	1163	1.21757e-8	1	2048	2.94036e-16	880	4	3	9	2	15	$2^{-64.0995}$
	16	1280	4.43607e-10	1	2048	2.94036e-16	1800	0	3	9	2	15	$2^{-64.1618}$
	17	1280	4.43607e-10	1	2048	2.94036e-16	3360	1	3	9	3	11	$2^{-64.2750}$
	18	1280	4.43607e-10	1	2048	2.94036e-16	7000	0	3	9	3	11	$2^{-64.2339}$
	19	1280	4.43607e-10	1	2048	2.94036e-16	16912	0	3	9	3	11	$2^{-64.0179}$
	20	1400	8.97680e-11	1	2048	2.94036e-16	41040	0	4	9	3	11	$2^{-64.0486}$
2^{-80}	6	768	8.76387e-6	1	2048	2.94036e-16	2	70	2	9	1	23	$2^{-80.1114}$
	7	818	2.22155e-6	1	2048	2.94036e-16	3	8	2	9	1	23	$2^{-80.1811}$
	8	818	2.22155e-6	1	2048	2.94036e-16	8	37	2	9	1	23	$2^{-80.2079}$
	9	879	1.96189e-7	1	2048	2.94036e-16	16	65	2	9	1	23	$2^{-80.0677}$
	10	971	1.96000e-7	1	2048	2.94036e-16	32	27	2	9	2	15	$2^{-80.0766}$
	11	971	1.96000e-7	1	2048	2.94036e-16	56	5	3	9	2	15	$2^{-80.2922}$
	12	1024	7.50000e-8	1	2048	2.94036e-16	108	4	3	9	2	15	$2^{-80.1033}$
	13	1088	4.44094e-8	1	2048	2.94036e-16	228	0	3	9	2	15	$2^{-80.3182}$
	14	1119	2.60130e-8	1	2048	2.94036e-16	488	0	3	9	2	15	$2^{-80.6946}$
	15	1163	1.21757e-8	1	2048	2.94036e-16	1000	0	3	9	2	15	$2^{-80.8264}$
	16	1280	4.43607e-10	1	2048	2.94036e-16	2040	0	3	9	2	15	$2^{-80.3281}$
	17	1280	4.43607e-10	1	2048	2.94036e-16	3712	0	3	9	3	11	$2^{-80.1860}$
	18	1280	4.43607e-10	1	2048	2.94036e-16	7808	0	3	9	3	11	$2^{-80.1454}$
	19	1280	4.43607e-10	1	2048	2.94036e-16	16800	0	4	9	3	11	$2^{-80.0300}$
	20	1280	4.43607e-10	1	2048	2.94036e-16	37200	0	4	9	4	9	$2^{-80.2144}$
2^{-128}	6	788	3.86795e-6	1	2048	2.94036e-16	2	35	2	9	1	23	$2^{-128.5065}$
	7	818	2.22155e-6	1	2048	2.94036e-16	4	19	2	9	1	23	$2^{-128.5630}$
	8	840	1.48614e-6	1	2048	2.94036e-16	16	337	2	9	1	23	$2^{-128.1163}$
	9	879	1.96189e-7	1	2048	2.94036e-16	20	12	2	9	1	23	$2^{-128.2469}$
	10	971	1.96000e-7	1	2048	2.94036e-16	32	3	3	9	2	15	$2^{-128.5467}$
	11	971	1.96000e-7	1	2048	2.94036e-16	80	14	3	9	2	15	$2^{-128.1814}$
	12	1024	7.50000e-8	1	2048	2.94036e-16	144	8	3	9	2	15	$2^{-128.3335}$
	13	1088	4.44094e-8	1	2048	2.94036e-16	312	8	3	9	2	15	$2^{-128.1529}$
	14	1163	1.21757e-8	1	2048	2.94036e-16	564	0	3	9	2	15	$2^{-128.3322}$
	15	1163	1.21757e-8	1	2048	2.94036e-16	1376	0	3	9	2	15	$2^{-128.3895}$
	16	1221	2.00000e-9	1	2048	2.94036e-16	2704	0	3	9	2	15	$2^{-128.1938}$
	17	1221	2.00000e-9	1	2048	2.94036e-16	4864	0	3	9	3	11	$2^{-128.0086}$
	18	1280	4.43607e-10	1	2048	2.94036e-16	10048	0	3	9	3	11	$2^{-128.6698}$
	19	1280	4.43607e-10	1	2048	2.94036e-16	22600	0	4	9	3	11	$2^{-128.0054}$
	20	1400	8.97680e-11	1	2048	2.94036e-16	39456	0	4	9	3	11	$2^{-128.0301}$

D Other Experiments

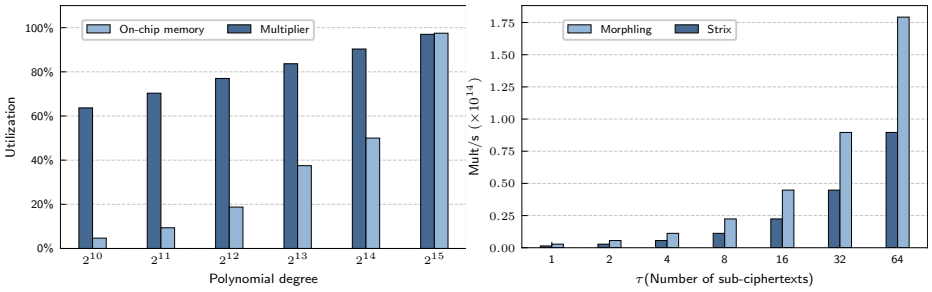
Table 9: Comparing SBS and EBS between our implementation and the implementation from [6]

Scheme	**Error**	**6**	**7**	**8**	**9**	**10**
SBS	2^{-40}	22.252 (×1)	53.340 (×1)	95.689 (×1)	191.206 (×1)	407.113 (×1)
	2^{-64}	31.191 (×1)	56.458 (×1)	107.061 (×1)	216.598 (×1)	449.839 (×1)
	2^{-80}	33.658 (×1)	75.352 (×1)	149.475 (×1)	307.290 (×1)	641.520 (×1)
	2^{-128}	41.093 (×1)	90.580 (×1)	179.797 (×1)	374.519 (×1)	794.830 (×1)
EBS	2^{-40}	26.725 (×1)	75.513 (×1)	146.352 (×1)	294.699 (×1)	604.757 (×1)
	2^{-64}	41.092 (×1)	75.864 (×1)	152.166 (×1)	309.411 (×1)	645.960 (×1)
	2^{-80}	50.093 (×1)	155.770 (×1)	282.680 (×1)	582.647 (×1)	1200.473 (×1)
	2^{-128}	54.233 (×1)	144.130 (×1)	290.206 (×1)	596.175 (×1)	1236.148 (×1)
SBS Original	2^{-40}	18.708 (×0.84)	44.771 (×0.84)	80.688 (×0.84)	156.700 (×0.82)	320.390 (×0.79)
	2^{-80}	26.602 (×0.79)	58.541 (×0.78)	121.980 (×0.82)	248.240 (×0.81)	512.820 (×0.80)
	2^{-128}	33.139 (×0.81)	72.112 (×0.80)	141.650 (×0.79)	286.210 (×0.76)	658.030 (×0.83)
EBS Original	2^{-40}	22.490 (×0.84)	62.254 (×0.82)	123.200 (×0.84)	250.530 (×0.85)	512.630 (×0.85)
	2^{-64}	33.858 (×0.82)	65.153 (×0.86)	127.830 (×0.84)	260.720 (×0.84)	547.440 (×0.85)
	2^{-80}	41.410 (×0.83)	127.080 (×0.82)	239.470 (×0.85)	489.230 (×0.84)	1013.100 (×0.84)
	2^{-128}	48.093 (×0.89)	120.620 (×0.84)	243.590 (×0.84)	501.900 (×0.84)	1038.500 (×0.84)

Table 10: MBS execution time for DFR 2^{-40}

Precision	4	5	6	7	8	9	10	11	12
Time	9.53 ms	12.09 ms	13.95 ms	25.62 ms	66.99 ms	126.27 ms	369.83 ms	714.42 ms	1.46 s
Precision	13	14	15	16	17	18	19	20	
Time	3.40 s	8.27 s	17.21 s	41.20 s	109.97 s	209.93 s	477.14 s	1205.93 s	

E Hardware Acceleration



(a) Hardware utilization (b) Accelerator performance

Fig. 4: Hardware acceleration estimation.

Bootstrapping remains the dominant performance bottleneck in TFHE and thus the primary target for hardware acceleration. State-of-the-art TFHE accelerators face structural challenges: (a) the exponential growth of the polynomial degree N with message precision, (b) massive bootstrapping keys in the range of hundreds of MB to several GB, and (c) limited intra-ciphertext parallelism. Our $\mathcal{R}_N$-module construction alleviates these issues, making it more hardware-friendly.

Hardware Efficiency. In current state-of-the-art hardware accelerator architectures [42,43,47,32,21], the feasible parameter choices for bootstrapping—such as the parallelism of FFT/NTT units and Multiply Accumulate units (MACs)—are primarily determined by the polynomial degree. Prior accelerators support low-precision bootstrapping, with polynomial degree typically ranging from 512 to 4096. Since the polynomial degree in classical bootstrapping algorithms grows exponentially with increasing bootstrapping precision, a 6-bit precision already corresponds to a polynomial degree of 2^{15}. To efficiently support higher-precision bootstrapping schemes, prior accelerators must widen the datapaths of the core functional units to accommodate longer polynomials. However, when operating at small parameters (e.g., $N = 1024$), accelerator utilization degrades significantly, with multiplier utilization around 66% and on-chip memory utilization around 5%.

In contrast, Our approach decouples N and q, and enhances precision by increasing the number of sub-ciphertexts while keeping the parameters of each sub-ciphertext fixed (e.g, a constant polynomial degree of 2048). Consequently, the accelerator can support bootstrapping operations at different precision levels without hardware architecture modifications, thereby substantially improving the hardware friendliness of bootstrapping algorithms.

Enhanced parallelism. To realize ciphertext-level parallelism, existing hardware accelerators implement multi-core architectures to fully pipeline bootstrapping operations [42,43]. There is a balance point between the number of parallel

compute cores and the available off-chip memory bandwidth: when the number of cores is below this balance point, overall performance scales approximately linearly with the number of cores; once the core count exceeds the balance point, additional cores yield diminishing returns. The diminishing gains arise because memory bandwidth becomes the bottleneck, causing frequent stalls in the compute pipelines due to massive off-chip memory accesses. For example, in the state-of-the-art TFHE accelerator Morphling, each compute core handles bootstrapping for a single ciphertext. Increasing the core count from 1 to 16 yields roughly a 16-fold performance improvement, but adding more than 16 cores offers only marginal gains (less than 25%).

In contrast, our approach decomposes a single ciphertext into τ sub-ciphertexts that can be processed in parallel, effectively shifting the performance–memory balance point by a factor of up to τ. Consequently, with the same off-chip memory bandwidth, the accelerator can achieve up to τ times higher throughput. This indicates that our approach can substantially enhance hardware performance with minimal additional hardware costs.

GPU friendliness. The parallelization of the bootstrap algorithm aligns naturally with the large-scale, concurrent thread architecture of GPUs. With enhanced parallelism in our scheme, more sub-ciphertext bootstrapping can be distributed across thread blocks and streaming processors for concurrent execution, thereby significantly improving hardware utilization and throughput. Moreover, optimizing the bootstrapping key size improves cache hit rates and reduces global memory access latency, further improving the execution efficiency of bootstrapping operations. GPU can achieve efficient parallel bootstrapping through batch processing and pipeline scheduling, meeting the high-performance demands of large-scale homomorphic encryption applications.

Overall, improving the computational parallelism of the bootstrapping algorithm, together with fixed polynomial degrees, enables hardware platforms such as ASICs and GPUs to better exploit their architectural advantages, leading to significant gains in performance, throughput, and energy efficiency of hardware acceleration.